

NATIONAL OPEN UNIVERSITY OF NIGERIA

FACULTY OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

SMART LECTURE REMINDER: A PROGRESSIVE WEB APPLICATION FOR AUTOMATED FACILITATION NOTIFICATIONS

BY

SULIEMAN ADAMU

NOU223098348

BEING A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE, FACULTY OF COMPUTING, NATIONAL OPEN UNIVERSITY OF NIGERIA IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF BACHELOR OF SCIENCE DEGREE IN COMPUTER SCIENCE.

JUNE, 2026

DECLARATION

I, SULIEMAN ADAMU, declare that this project work is the product of my own research and effort. All sources consulted have been adequately acknowledged and properly cited throughout this report. Any errors or shortcomings found herein are my own. This project has not been submitted, in part or in whole, to this department or any other institution for the award of a degree or any other qualification.

Signature:  Date: _____

DEDICATION

This project is dedicated to every student of the National Open University of Nigeria who has ever missed a facilitation session not out of carelessness, but because the system in place gave no warning. It is also dedicated to my family, whose patience, encouragement, and unwavering support carried me through this programme.

APPROVAL PAGE

This project, titled "Smart Lecture Reminder: A Progressive Web Application for Automated Facilitation Notifications," has been read and approved as meeting the requirements for the Bachelor of Science degree in Computer Science, Department of Computer Science, Faculty of Computing, National Open University of Nigeria.

_____	_____
Dr. Ado Abubakar Rogo	DATE
Supervisor	
_____	_____
Dr. Francis Bukie Osang	DATE
Head of Department	
_____	_____
[Name of External Examiner]	DATE
External Examiner	

ACKNOWLEDGMENT

I am most grateful to God Almighty for granting me the strength, wisdom, and grace to complete this programme and this project. I am deeply thankful to my parents and family for their unwavering support, encouragement, and sacrifices throughout my academic journey.

I wish to express my sincere gratitude to my supervisor, Dr. Ado Abubakar Rogo, for the guidance, patience, and constructive feedback provided throughout the development of this project. I also acknowledge the Vice-Chancellor, the Dean of the Faculty of Computing, the Head of Department, and all the lecturers of the Department of Computer Science, National Open University of Nigeria, for their support and the knowledge imparted throughout this programme.

Finally, I appreciate my friends and course-mates who provided encouragement, feedback, and moral support during the course of this work.

ABSTRACT

This study presents the design and implementation of the Smart Lecture Reminder, a Progressive Web Application developed to address the absence of automated facilitation reminders at the National Open University of Nigeria (NOUN), where students rely on a static PDF timetable with no push notifications and no cancellation alerts. The system was built using React.js, Supabase (PostgreSQL, authentication, real-time sync), and the Web Push API with VAPID authentication via Supabase Edge Functions and Supabase Cron, with no dependency on Firebase or any paid service. It supports three roles: Administrators manage the timetable, TMA deadlines, and user accounts; Lecturers reschedule, cancel, and post announcements; and Students are auto-enrolled via in-browser PDF.js extraction of their registration slip, receive push reminders at 30, 15, and 5 minutes before each lecture, and self-track attendance. Seven of ten scope items were fully verified end-to-end. The system is live at https://smart-lecture-reminder-noun.netlify.app.

TABLE OF CONTENTS

TITLE....1

DECLARATION....2

DEDICATION....3

APPROVAL PAGE....4

ACKNOWLEDGMENT....5

ABSTRACT....6

LIST OF TABLES....8

LIST OF FIGURES....10

CHAPTER ONE: INTRODUCTION....13

1.1 Background of the Study....14

1.2 Statement of the Problem....15

1.3 Aim and Objectives....16

1.4 Scope and Limitations....17

1.5 Significance of the Project....18

1.6 Project Document Organization....19

1.7 Definition of Terms....19

CHAPTER TWO: LITERATURE REVIEW....22

2.1 Introduction....22

2.2 Review of Fundamental Concepts....22

2.3 Review of Related Works....25

2.4 General Discussion....27

2.5 Chapter Summary....28

CHAPTER THREE: RESEARCH METHODOLOGY....29

3.1 Design Consideration....29

3.2 Design Architecture.....29

3.3 System Design.....31

3.4 UML Diagrams.....43

3.5 Chapter Summary.....52

CHAPTER FOUR: IMPLEMENTATION AND EVALUATION.....53

4.1 System Hardware Requirement.....53

4.2 System Software Requirement.....53

4.3 Data Source.....64

4.4 Implementation Procedure.....67

CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS.....68

5.1 Introduction.....68

5.2 Summary.....68

5.3 Conclusion.....70

5.4 Recommendations and Future Work.....71

REFERENCES.....73

APPENDIX A SOURCE CODE LISTINGS.....75

APPENDIX B SYSTEM INSTALLATION AND DEPLOYMENT GUIDE.....78

APPENDIX C USER MANUAL.....80

APPENDIX D TEST RESULTS SUMMARY.....82

APPENDIX E SAMPLE DATA.....83

LIST OF TABLES

Table 1.1 The Ten Locked In-Scope Feature Areas.....17

Table 2.1 Summary of Related Works Review.....27

Table 3.1 Functional Requirements of the Proposed System.....32

Table 3.2 Non-Functional Requirements of the Proposed System.....33

Table 3.3 users Table Database Schema.....34

Table 3.4 courses Table Database Schema.....34

Table 3.5 lecturer_courses Table Database Schema.....35

Table 3.6 lectures Table Database Schema.....36

Table 3.7 enrollments Table Database Schema.....36

Table 3.8 attendance Table Database Schema.....38

Table 3.9 tma_deadlines Table Database Schema.....38

Table 3.10 notifications Table Database Schema.....39

Table 3.11 app_settings Table Database Schema.....39

Table 4.1 Development Tools and Libraries.....53

Table 4.2 Unit Test Cases and Results.....64

Table 4.3 Integration Test Cases and Results.....65

Table 4.4 Definition of Done Submission Checklist Status.....66

LIST OF FIGURES

Figure 3.4.1 Use Case Diagram of the Smart Lecture Reminder System.....43

Figure 3.4.2 Entity Relationship Diagram (ERD).....45

Figure 3.4.3 Activity Diagram Student Onboarding and Course Enrollment.....47

Figure 3.4.4 Sequence Diagram Push Notification Delivery.....48

Figure 3.4.5 System Architecture Diagram.....50

Figure 4.1 Login Page.....60

Figure 4.2 Registration Page with Role Selector.....60

Figure 4.3 Student Onboarding Wizard.....60

Figure 4.4 Student Dashboard.....60

Figure 4.5 Calendar Month, Week, and Day Views.....60

Figure 4.6 Lecture Detail Bottom Sheet.....61

Figure 4.7 Admin Dashboard and Lecture CRUD.....61

Figure 4.8 Admin CSV Bulk Import.....61

Figure 4.9 Admin TMA Deadline Management.....61

Figure 4.10 Admin Pending Lecturer Approval.....61

Figure 4.11 Admin User Management and Semester Settings.....62

Figure 4.12 Lecturer Dashboard.....62

Figure 4.13 In-App Notification Panel.....62

Figure 4.14 Push Notification Received on Device.....62

Figure 4.15 PWA Install Flow and Offline Mode.....63

Figure 4.16 Settings Page (Per Role).....63

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

NOUN was established by the National Open University of Nigeria Act 2002 and is headquartered in Jabi, Abuja. As Nigeria's dedicated open and distance learning institution, NOUN delivers instruction predominantly through self-instructional course materials supplemented by scheduled facilitation sessions, which serve as the primary live, synchronous component of the learning experience. Each semester's facilitation period runs for approximately eight weeks, during which registered facilitators deliver scheduled Zoom sessions for each course (Laudon and Laudon, 2025).

At present, the only official mechanism for communicating the facilitation schedule to students is a static PDF timetable downloadable from the eLearn portal. This document is undifferentiated by student programme or registered courses, requiring each student to manually search the document for their relevant sessions each week. In the absence of an official notification mechanism, students and facilitators have organically formed informal WhatsApp and Telegram groups to broadcast reminders, cancellations, and rescheduling notices a workaround that is unreliable, inconsistently maintained, and excludes any student who is not a member of the relevant group (Adewale, Adaora, and Uchenna, 2021).

The emergence of Progressive Web Application technology, combined with the Web Push API and VAPID authentication standardized in RFC 8030 and RFC 8292 respectively, has made it technically and financially feasible to build a browser-installable application capable of delivering operating-system-level push notifications without requiring a native mobile application or a paid notification service such as Firebase Cloud Messaging. This technical

maturity, reached only in the years immediately preceding this project (with Apple completing Web Push support in Safari on iOS 16.4 in 2023), directly motivates the architecture adopted in this project.

1.2 Statement of the Problem

The administration of facilitation scheduling and student notification at NOUN continues to rely on a static document and informal communication channels. A detailed examination of the existing system reveals the following specific problems:

- No push notification capability of any kind exists within the official NOUN infrastructure for upcoming facilitation sessions.
- The facilitation timetable is a static PDF with no mechanism for dynamic updates; any rescheduling requires the student to re-download and manually compare the document.
- There is no official mechanism to notify enrolled students when a facilitator cancels or reschedules a session.
- The timetable is not personalized it lists every course across every programme and level in a single undifferentiated document.
- The eLearn portal, including the timetable, is entirely inaccessible without an active internet connection.
- There is no official system for surfacing Tutor-Marked Assignment (TMA) deadlines alongside the facilitation schedule.
- Facilitators have no official, in-system channel for posting session-specific announcements or confirming the availability of session recordings.

These deficiencies collectively contribute to a pattern of missed facilitation sessions, academic disadvantage for affected students, and an inequitable reliance on informal communication channels that not every student has access to (O'Brien and Marakas, 2020).

1.3 Aim and Objectives

The aim of this project is to design and implement a web-based Progressive Web Application that automates facilitation session reminders for students of the National Open University of Nigeria, eliminating dependence on manual timetable checking and unofficial communication channels. The specific objectives are:

1. To analyse the requirements and limitations of the existing facilitation notification and scheduling process at NOUN.
2. To design a comprehensive three-role system architecture, relational database schema, and user interface, modelled using Unified Modelling Language (UML) diagrams.
3. To implement the designed system using React.js for the frontend, Supabase for backend services, and the Web Push API with VAPID authentication for notification delivery.
4. To implement automated and instant push notification delivery using Supabase Edge Functions and Supabase Cron, without dependence on any paid third-party notification service.
5. To test the implemented system using unit and integration testing methodologies, and to verify functionality through end-to-end testing across all three user roles.
6. To deploy the completed system to a live, publicly accessible production environment and evaluate it against the locked project scope.

1.4 Scope and Limitations

The scope of this project is explicitly and deliberately bounded to ten locked feature areas, defined and frozen prior to the final development push to ensure the project could be completed and submitted within the available timeframe. These ten feature areas, reproduced in Table 1.1, constitute the complete and only scope of the implemented system. No feature outside this list was built before submission.

Table 1.1: The Ten Locked In-Scope Feature Areas

#	In-Scope Feature Area	Description
1	Three-role authentication	Admin / lecturer / student with RBAC; pending-approval flow for lecturers; forgot password
2	Personalized lecture calendar	Month, Week, Day views; shows only enrolled courses; LIVE badge; cancelled/rescheduled states
3	Course enrollment via PDF.js	In-browser extraction of course codes from uploaded registration slip; manual fallback
4	Automated Web Push reminders (VAPID)	30, 15, and 5 minutes before each lecture; 24 hours and 1 hour before each TMA deadline
5	Instant push alerts	Cancellation, reschedule, announcement posted, recording added
6	Lecturer panel	Reschedule lecture, cancel with reason, post 280-character announcement, add recording URL
7	Admin panel	Full lecture CRUD, CSV bulk import, TMA deadline management, user management, semester settings, pending lecturer approval
8	Attendance self-tracking	Mark attended/missed/excused per past lecture; attendance summary with progress bar on course cards
9	TMA deadlines in calendar	Gold TMA blocks shown alongside lectures for enrolled courses; NextUpCard TMA variant
10	Progressive Web Application (PWA)	Installable on Android/iOS, offline-capable via service worker, update banner

The system explicitly excludes the following features, which were considered during the design phase but deliberately deferred as Future Work to keep the project achievable within the available time:

- Dark mode toggle
- Weekly digest push notification (Monday 8am summary)

- Recurring lecture templates (auto-generated repeating sessions)
- Study material links per lecture
- Attendance data export (CSV)
- Past lectures archive (recordings library)
- NOUN study centre filter

The system is optimized for modern web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge) and for Apple Safari on iOS 16.4 and above, the minimum version at which Apple implemented Web Push API support. Devices running an earlier version of iOS Safari can use the application normally but will receive in-app notifications only, without operating-system-level push delivery.

1.5 Significance of the Project

From an institutional standpoint, the Smart Lecture Reminder addresses a documented, recurring problem affecting the entire NOUN student population each semester, with the potential to materially improve facilitation attendance and, by extension, student academic outcomes. The system's three-role access control, enforced independently at both the application layer and the database layer through Supabase Row Level Security, provides defense-in-depth security guarantees that the current static-document approach does not offer.

From an academic and technical standpoint, this project demonstrates that a Progressive Web Application combining real-time data synchronization, in-browser document processing, and a serverless push notification pipeline can be designed, implemented, and deployed entirely on free-tier cloud infrastructure, without dependence on Firebase, Google Cloud Messaging, or any paid third-party service. This makes the architecture documented in this report a viable,

replicable blueprint for other distance learning institutions facing similar notification and scheduling challenges, particularly within resource-constrained academic settings.

1.6 Project Document Organization

This project report is organized into five chapters. Chapter One introduces the study, covering the background, problem statement, aim and objectives, scope and limitations, significance, and definition of terms. Chapter Two reviews relevant literature on the foundational concepts and related works underpinning the project. Chapter Three presents the system analysis and design, including the functional and non-functional requirements, database specification, and UML diagrams. Chapter Four documents the system implementation and the results of unit and integration testing. Chapter Five presents the summary, conclusion, and recommendations for future work.

1.7 Definition of Terms

- Facilitation: The NOUN institutional term for a scheduled, synchronous virtual lecture session delivered by a registered facilitator via Zoom during the approximately eight-week facilitation window of each semester.
- Facilitator (Lecturer): An academic staff member of NOUN assigned to deliver facilitation sessions for one or more courses. Within the Smart Lecture Reminder, facilitators hold the lecturer role.
- Tutor-Marked Assignment (TMA): A graded assignment administered through the NOUN eLearn portal, typically scheduled at three points (TMA 1, 2, 3) within a semester.
- Progressive Web Application (PWA): A web application that employs the Service Worker API, the Web App Manifest, and the Push API to deliver an experience

comparable to a native mobile application, including offline access, home-screen installation, and push notification support.

- Push Notification: A message transmitted from a server to a client device and displayed by the operating system as a notification, even when the originating application is not open in the foreground.
- VAPID (Voluntary Application Server Identification): A security protocol, specified in RFC 8292, that enables a push notification server to sign its requests with a private key so that a browser push service can verify the sender's identity without a third-party intermediary.
- Row Level Security (RLS): A PostgreSQL feature, exposed by Supabase, that restricts which rows a given database user can read, insert, update, or delete, based on security policies defined at the table level.
- Role-Based Access Control (RBAC): A security model that restricts system access and permissions based on a user's assigned role.
- Supabase: An open-source Backend-as-a-Service platform built on PostgreSQL, providing authentication, a relational database, real-time data subscriptions, and Row Level Security.
- Service Worker: A JavaScript file that runs in a background browser thread, independent of any specific web page, used for caching, background synchronization, and the receipt of push notification events.
- CRUD: Create, Read, Update, and Delete the four basic operations performed on persisted data.
- UML: Unified Modelling Language a standardized visual language for modelling system structure and behaviour.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter presents a structured review of literature relevant to this project, examining foundational concepts that underpin the system before surveying directly related works. All sources are cited in accordance with APA 6th Edition referencing conventions.

2.2 Review of Fundamental Concepts

This section examines the key theoretical concepts and technological foundations that underpin the Smart Lecture Reminder system. The review is organized into seven sub-sections covering management information systems, web-based information systems and Progressive Web Applications, React.js as a frontend framework, Supabase as a back-end-as-a-service platform, PostgreSQL with Row Level Security and real-time data synchronization, the Web Push API with VAPID authentication, and software development methodologies.

2.2.1 Management Information Systems

A Management Information System (MIS) is broadly defined as an integrated, computer-based system designed to collect, process, store, and disseminate information useful for organizational decision-making (Laudon & Laudon, 2025). In an academic scheduling context, an effective MIS must prioritize timely information delivery, personalisation to the individual user, and reliability under intermittent connectivity (O'Brien & Marakas, 2020) three properties absent from NOUN's current static-document approach to facilitation scheduling.

2.2.2 Web-Based Information Systems and Progressive Web Applications

Web-based information systems deliver functionality through a browser, offering platform independence and centralized, server-side data management (Connolly & Begg, 2015). Progressive Web Applications extend this model by employing the Service Worker API, the Web App Manifest, and the Push API to provide capabilities traditionally exclusive to native mobile applications, including offline access, home-screen installation, and operating-system-level push notifications, all without requiring distribution through an application store. Biorn-Hansen, Majchrzak, and Gronli (2017) compared native, hybrid, and progressive web application architectures and concluded that PWAs offer the most favourable balance between development cost and delivered capability for cross-platform applications requiring push notifications and offline support a finding directly relevant to a single-codebase application intended to serve a student population using heterogeneous Android, iOS, and desktop devices.

2.2.3 React.js as a Frontend Framework

React.js, released by Meta (formerly Facebook) in 2013, uses a component-based architecture and a virtual DOM reconciliation mechanism that makes it well-suited for applications with frequent, data-driven user interface updates, such as a live calendar that must reflect real-time changes pushed from a backend database (Dodds, 2020). For this project, React is paired with React Router v6 for client-side routing and the React Context API for global state management, avoiding the additional dependency footprint of a third-party state management library.

2.2.4 Supabase as a Backend-as-a-Service Platform

Supabase is an open-source Backend-as-a-Service platform built on PostgreSQL. It provides Authentication (email and password, with extensibility to OAuth providers), a fully relational PostgreSQL database with complete SQL support, Realtime data subscriptions over

WebSockets, and Row Level Security (RLS) for fine-grained, database-enforced access control, all without requiring the management of server infrastructure (Supabase, 2024).

Supabase was selected in preference to Google Firebase for three principal reasons relevant to this project. First, Firebase Cloud Functions the component that would be required to schedule and dispatch automated push notifications cannot be deployed on Firebase's free Spark plan; it requires the paid, pay-as-you-go Blaze plan, even for low-volume scheduled triggers. Second, Firestore, Firebase's NoSQL document database, imposes meaningful limitations on complex, multi-field relational queries of the kind required to derive a personalized student calendar from normalized enrollment and lecture records. Third, Supabase's relational PostgreSQL foundation, combined with Row Level Security, provides a stronger and more auditable data access guarantee for a three-role academic system than a document-oriented alternative.

2.2.5 PostgreSQL, Row Level Security, and Real-Time Data Synchronization

PostgreSQL is a mature, open-source object-relational database management system that enforces predefined schemas, supports ACID-compliant transactions, and provides sophisticated query capabilities including multi-table joins and aggregate functions (Sadalage & Fowler, 2013). Supabase exposes PostgreSQL's Row Level Security feature as a first-class access control mechanism, allowing security policies to be defined directly on database tables for example, restricting a lecturer's update permission on the lectures table to only the rows where their own user ID matches the assigned facilitator. This database-level enforcement operates independently of, and as a defense-in-depth complement to, application-layer route protection.

Real-time synchronization between the database and connected clients is achieved through Supabase Realtime, which broadcasts row-level INSERT, UPDATE, and DELETE events over

WebSocket connections using PostgreSQL's native change data capture mechanism. This capability underpins the requirement that a change made by an administrator or lecturer such as creating a new lecture or rescheduling an existing one must be reflected immediately in all connected students' calendar views without requiring a manual page refresh.

2.2.6 The Web Push API and VAPID Authentication

The Web Push API, standardized in RFC 8030 (Thomson, Damaggio, & Beverloo, 2016), specifies the protocol for generic event delivery over HTTP, enabling a web application to register to receive push messages from a server even when the application is not actively open in the browser. The VAPID (Voluntary Application Server Identification) extension, specified in RFC 8292 (Thomson & Beverloo, 2017), adds an application server identification mechanism that allows the push services operated by browser vendors to authenticate the sender of a notification using a public/private key pair, without requiring registration with any proprietary intermediary such as Firebase Cloud Messaging. Apple completed Web Push support in Safari on iOS 16.4 in 2023, finally closing the last significant cross-platform gap and making a fully Firebase-independent push architecture viable across Android, iOS, and desktop browsers a development that arrived sufficiently recently to directly shape the architectural decisions of this project.

2.2.7 Software Development Methodologies

The Agile software development model was adopted for this project. Agile is an iterative, incremental approach that emphasizes early delivery of working software and the ability to adapt to changing requirements as they are discovered during development (Beck et al., 2001). The Waterfall model was not selected because its sequential, phase-locked structure would have been poorly suited to the kind of iterative refinement required for the push notification

pipeline and the calendar rendering logic, both of which required adjustment as functional gaps were discovered during implementation.

2.3 Review of Related Works

Adewale, Adaora, and Uchenna (2021) conducted a structured survey of distance learning experiences across five NOUN study centres and found that session attendance was the most frequently cited academic challenge, with over sixty percent of respondents reporting having missed at least one facilitation session per semester due to inadequate notification a finding that directly substantiates the problem this project addresses.

Agbo, Nwosu, Eze, and Okafor (2022) developed a lecture attendance and notification system for a Nigerian federal university using a native Android application and Firebase Cloud Messaging, reporting a 34 percent improvement in attendance during a pilot period. However, their system required a Firebase Blaze paid billing account for the Cloud Functions that scheduled notification dispatch, and excluded students without Android devices both limitations directly addressed by the cross-platform, Firebase-independent architecture adopted in this project.

Oyelaran and Okike (2019) examined the feasibility of SMS-based academic notification systems for Nigerian university students and found that while delivery was broadly reliable, SMS could not carry contextual action data such as a clickable Zoom meeting link, and incurred a per-message cost at scale. Nwachukwu and Emeka (2020) implemented a WhatsApp Business API broadcast system for lecture reminders, reporting improved attendance but noting that the approach depended on student group membership and risked violating WhatsApp's policies against mass automated broadcasting.

Biorn-Hansen, Majchrzak, and Gronli (2017) conducted a comparative technical analysis of native, hybrid, and progressive web application architectures, concluding that PWAs offer the most favourable trade-off between development cost and cross-platform push notification and offline capability directly informing the technology selection for this project.

Table 2.1: Summary of Related Works Review

Author(s) & Year	System	Stack	Strengths	Weaknesses
Adewale et al. (2021)	Survey of distance learning challenges at NOUN	Survey/Empirical	Confirmed session attendance as the most cited challenge	Not implementation-focused
Agbo et al. (2022)	Lecture attendance and notification system	Android, Firebase Cloud Messaging	34% reported attendance improvement	Requires paid Firebase Blaze billing plan; Android-only
Oyelaran & Okike (2019)	SMS-based academic notification feasibility	SMS Gateway	Reliable delivery	No action data (e.g. meeting links); per-message cost
Nwachukwu & Emeka (2020)	WhatsApp Business API broadcast reminders	WhatsApp Business API	Improved attendance reported	Requires group membership; violates broadcast policy at scale
Biorn-Hansen et al. (2017)	Comparative analysis of native, hybrid, and PWA architectures	Comparative Review	PWAs offer best cost/capability trade-off for push + offline	Review only; not a deployed system

2.4 General Discussion

The reviewed literature confirms both the existence of the problem this project addresses and the technical feasibility of the proposed solution. Adewale et al. (2021) provide direct empirical evidence that NOUN students experience a recurring, documented attendance problem attributable to inadequate notification infrastructure. The SMS and WhatsApp-based approaches reviewed (Oyelaran & Okike, 2019; Nwachukwu & Emeka, 2020) demonstrate that informal or low-fidelity notification channels have already been explored in the Nigerian context but are limited by cost, missing action data, or platform policy risk. The native Android

approach of Agbo et al. (2022) demonstrates the underlying notification concept works, but at the cost of a paid billing dependency and a single-platform limitation, both of which are directly resolved by the PWA and Web Push/VAPID architecture adopted in this project. No prior work reviewed integrates NOUN's specific facilitation context with a fully free, cross-platform push notification pipeline, in-browser course registration slip processing, and a three-role database-enforced access model within a single system constituting the original contribution of this project.

2.5 Chapter Summary

This chapter reviewed seven foundational concepts underpinning the project management information systems, progressive web applications, React.js, Supabase, PostgreSQL with Row Level Security and real-time synchronization, the Web Push API with VAPID authentication, and Agile development methodology and five directly related works. The review established both the empirical existence of the attendance problem at NOUN and the absence of any prior system combining a fully free, cross-platform notification architecture with NOUN's specific facilitation context. The next chapter presents the system analysis and design.

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Design Consideration

This chapter presents the analytical and design framework that guided the development of the Smart Lecture Reminder. It describes the analysis of the existing facilitation notification process at NOUN, the design of the proposed system, the functional and non-functional requirements, the database schema, the justification for the chosen technology stack, and five UML diagrams modelling the system's structure and behaviour. Each UML diagram is presented in this chapter as a clearly labelled placeholder containing the corresponding Mermaid diagram code; the rendered diagram image should be generated from this code and inserted at the indicated figure position.

3.2 Design Architecture

This section presents the analysis of the existing facilitation notification and scheduling process at the National Open University of Nigeria (NOUN). The analysis was conducted through document review and structured observation, and is organized into three sub-sections: the method of analysis adopted, an investigation of the current system, and an identification of the specific problems inherent in that system.

3.2.1 Method of System Analysis

The system analysis phase was conducted through document review and structured observation. Document review involved a study of NOUN's published facilitation timetable format, the eLearn portal's course registration and Zoom-link distribution process, and the documented student attendance challenges reported in the literature reviewed in Chapter Two. Structured observation of the existing facilitation notification process specifically, the cycle of

downloading a static PDF timetable, manually identifying relevant courses, and relying on informal WhatsApp or Telegram groups for change notifications provided the baseline understanding of the gap this project addresses.

3.2.2 Investigation and Analysis of the Existing System

The existing facilitation notification process at NOUN is entirely manual from the student's perspective. A student logs into the eLearn portal, downloads a static PDF timetable covering every course across every programme and level, and must manually identify the rows relevant to their own registered courses. Zoom meeting links for each session are accessed by navigating to the corresponding course page on eLearn. When a facilitator needs to reschedule or cancel a session, the only mechanism for communicating this change is an informal post in a WhatsApp or Telegram group that the student must already be a member of. There is no notification of any kind delivered to a student's device, and the timetable document provides no indication of which entries have changed since the previous version was downloaded.

3.2.3 Problems with the Existing System

- No push notification of any kind for upcoming facilitation sessions.
- Static PDF timetable with no mechanism for dynamic, differential updates.
- No official notification when a facilitator cancels or reschedules a session.
- No personalisation the timetable lists every course across every programme in one undifferentiated document.
- No offline access to the schedule; the eLearn portal requires an active internet connection at all times.
- No visibility of Tutor-Marked Assignment deadlines alongside the facilitation schedule.

- No official channel for facilitators to post session-specific announcements or confirm recording availability.
- No record of student attendance of any kind, self-reported or otherwise.

3.3 System Design

This section describes the design of the proposed Smart Lecture Reminder system, translating the problems identified in Section 3.2 into a structured technical solution. The section is organized into six sub-sections covering the description of the proposed system, the development model, functional and non-functional requirements, the database specification across nine tables, the technology stack selection, and the UML modelling approach.

3.3.1 Description of the Proposed System

The proposed Smart Lecture Reminder is a web-based, cloud-hosted Progressive Web Application that directly addresses every problem identified in Section 3.2.3, within the explicit boundaries of the ten locked in-scope feature areas reproduced in Table 1.1. The system automatically enrolls a student in their registered courses by extracting course codes from their uploaded course registration slip using in-browser PDF processing; displays a personalized calendar showing only the lectures and TMA deadlines for those enrolled courses; sends automated push notifications 30, 15, and 5 minutes before each lecture and 24 hours and 1 hour before each TMA deadline; sends instant push notifications when a lecturer cancels, reschedules, posts an announcement, or adds a recording link; remains usable offline through service worker caching; and is installable on Android and iOS as a home-screen application.

The system supports three user roles. Administrators manage the master semester timetable, the course catalogue, TMA deadlines, user accounts, and the approval of pending lecturer registrations. Lecturers (Facilitators) manage only the lectures assigned to them rescheduling,

cancelling, posting announcements, and adding recording links. Students complete an Onboarding process that auto-enrolls them in their registered courses, after which they view a personalized calendar, receive automated and instant push reminders, and self-report their attendance for past lectures.

3.3.2 Development Model and Justification

The Agile software development model was adopted for this project, structured across a fourteen-week implementation timeline. Agile was selected over Waterfall because several components of the system particularly the push notification reminder windows, the calendar rendering logic for overlapping events, and the PDF course code extraction pattern required iterative refinement as functional gaps were discovered during development. The fourteen-week timeline was organized into discrete, sequential phases: project setup and database schema design (Weeks 1–2), authentication and UI primitives (Weeks 3–4), Onboarding and real-time data integration (Weeks 5–6), the Admin and Lecturer panels (Weeks 7–8), attendance tracking (Week 9), the push notification pipeline (Week 10), the notification centre (Week 11), Progressive Web Application support (Week 12), testing (Week 13), and production deployment and documentation (Week 14).

3.3.3 Functional and Non-Functional Requirements

The functional requirements of the Smart Lecture Reminder, derived directly from the ten locked in-scope feature areas, are presented in Table 3.1.

Table 3.1: Functional Requirements of the Proposed System

Req. ID	Functional Requirement	Priority
FR-01	The system shall allow users to register with an email and password and select a role (student or lecturer).	High
FR-02	The system shall authenticate users with email and password before granting access, and shall support a forgot-password flow.	High
FR-03	The system shall enforce role-based access for Admin, Lecturer, and Student, with	High

Req. ID	Functional Requirement	Priority
	database-level enforcement via Row Level Security.	
FR-04	The system shall set newly registered lecturer accounts to a pending status until approved by an administrator.	High
FR-05	The system shall display a personalized calendar (Month, Week, and Day views) showing only the lectures and TMA deadlines for a student's enrolled courses.	High
FR-06	The system shall extract course codes from a student's uploaded course registration slip using in-browser PDF processing, and present them as a pre-ticked enrollment confirmation list.	High
FR-07	The system shall provide a manual course selection fallback when automatic PDF extraction is not possible (e.g. for a scanned image).	Medium
FR-08	The system shall send automated push notifications 30, 15, and 5 minutes before each lecture to enrolled, opted-in students.	High
FR-09	The system shall send automated push notifications 24 hours and 1 hour before each TMA deadline to enrolled, opted-in students.	High
FR-10	The system shall send an instant push notification to enrolled students when a lecturer cancels or reschedules a lecture.	High
FR-11	The system shall send an instant push notification to enrolled students when a lecturer posts an announcement or adds a recording link.	Medium
FR-12	The system shall allow an administrator to create, read, update, and delete lectures, including bulk import via CSV.	High
FR-13	The system shall allow an administrator to manage TMA deadlines, user accounts, semester settings, and pending lecturer approvals.	High
FR-14	The system shall allow a lecturer to reschedule or cancel only the lectures assigned to them, and to post an announcement or add a recording link.	High
FR-15	The system shall allow a student to self-report attendance (attended, missed, or excused) for a past lecture.	Medium
FR-16	The system shall be installable as a Progressive Web Application and remain functional, with cached schedule data visible, while offline.	High

The non-functional requirements are presented in Table 3.2.

Table 3.2: Non-Functional Requirements of the Proposed System

Req. ID	Non-Functional Requirement	Category
NFR-01	The system shall load the dashboard within 3 seconds on a standard broadband connection.	Performance
NFR-02	The system shall be accessible on modern browsers (Chrome, Firefox, Edge) and Safari on iOS 16.4 and above without plugin installation.	Usability
NFR-03	The system shall be mobile-first and display correctly on screen widths from 360px and above.	Usability

Req. ID	Non-Functional Requirement	Category
NFR-04	All data transmitted between the client and Supabase shall be encrypted using TLS.	Security
NFR-05	The system shall enforce Supabase Row Level Security policies on every table to prevent unauthorized data access independent of application-layer checks.	Security
NFR-06	The system shall be available with high uptime by leveraging Supabase's and Netlify's managed infrastructure.	Reliability
NFR-07	The push notification dispatch function shall execute on a five-minute schedule without requiring any paid billing account.	Cost / Scalability
NFR-08	The system user interface shall be intuitive enough for a non-technical student or lecturer to use without formal training.	Usability

3.3.4 Database Specification

The Smart Lecture Reminder uses Supabase PostgreSQL as its database, organized into nine relational tables, each enforced with Row Level Security policies. The schema is normalized to eliminate data redundancy while supporting the efficient relational queries required to derive a personalized student calendar from enrollment and lecture records.

The users table extends Supabase Auth and stores profile data for all three roles in a single table, distinguished by the role field. Table 3.3 presents the schema.

Table 3.3: users Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	References auth.users(id) Supabase Auth identifier
email	TEXT	User's email address
full_name	TEXT	User's full name
matric_number	TEXT	Student's matriculation number (students only)
programme	TEXT	Student's programme of study (students only)
level	INTEGER	Student's academic level (students only)
study_centre	TEXT	Student's study centre (students only)
faculty	TEXT	Lecturer's faculty (lecturers only)
department	TEXT	Lecturer's department (lecturers only)
role	TEXT	student | lecturer | admin

Column Name	Data Type	Description
status	TEXT	active | pending | suspended
push_subscriptions	JSONB	Array of Web Push subscription objects (multi-device)
notification_preferences	JSONB	Per-type notification toggle preferences
Onboarding_complete	BOOLEAN	Whether the student has completed the Onboarding wizard
created_at	TIMESTAMPTZ	Account creation timestamp

The courses table is the master record for each course offered in a given semester and academic session, created and managed exclusively by administrators. Table 3.4 presents the schema.

Table 3.4: courses Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
course_code	TEXT UNIQUE	NOUN course code, e.g. CIT 403
course_title	TEXT	Full course title
credit_units	INTEGER	Credit unit value of the course
programme	TEXT	Programme the course belongs to
level	INTEGER	Academic level the course is offered at
semester	TEXT	first | second
academic_session	TEXT	e.g. 2026_1
color_hex	TEXT	Display colour assigned to the course on the calendar
enrollment_count	INTEGER	Cached count of enrolled students
created_at	TIMESTAMPTZ	Record creation timestamp

The lecturer courses table is a junction table associating a lecturer with the courses they have been assigned by an administrator for a given academic session. Table 3.5 presents the schema.

Table 3.5: lecturer courses Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
lecturer_id	UUID (FK)	References users(id) the assigned lecturer
course_id	UUID (FK)	References courses(id)
academic_session	TEXT	The academic session of the assignment

Column Name	Data Type	Description
assigned_at	TIMESTAMPTZ	Timestamp of assignment by an administrator

The lectures table records each scheduled facilitation session, including Zoom meeting details, status, and a JSONB flag tracking which automated reminder intervals have already been dispatched. Table 3.6 presents the schema.

Table 3.6: lectures Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
course_id	UUID (FK)	References courses(id)
course_code / course_title / color_hex	TEXT	Denormalized course display fields
facilitator_id	UUID (FK)	References users(id) the assigned lecturer
facilitator_name	TEXT	Denormalized facilitator display name
venue	TEXT	Session venue (typically "Online")
meeting_platform / meeting_web_url / meeting_deep_link / meeting_id	TEXT	Zoom meeting details, deep link, and extracted meeting ID
recording_url	TEXT	Recording link added by the lecturer after the session
announcement / announcement_sent	TEXT / BOOLEAN	Lecturer's 280-character session announcement and dispatch flag
start_time / end_time	TIMESTAMPTZ	Scheduled start and end time of the lecture
status	TEXT	scheduled | cancelled | rescheduled | completed
cancel_reason	TEXT	Reason supplied by the lecturer when cancelling
rescheduled_from	TIMESTAMPTZ	Original start time, retained when rescheduled
notifications_sent	JSONB	Flags tracking whether the 30/15/5-minute reminders have fired
semester / academic_session	TEXT	Semester and session the lecture belongs to
created_at / updated_at	TIMESTAMPTZ	Record creation and last update timestamps

The enrollments table associates a student with a course for a specific academic session, with an expired flag preserving historical enrollment data after a semester concludes. Table 3.7 presents the schema.

Table 3.7: enrollments Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
user_id	UUID (FK)	References users(id) the enrolled student
course_id	UUID (FK)	References courses(id)
course_code	TEXT	Denormalized course code
semester / academic_session	TEXT	The semester and session of enrollment
custom_color	TEXT	Student's personalized override colour for this course
reminder_override	INTEGER[]	Optional per-course override of reminder intervals
expired	BOOLEAN	True once the enrollment's academic session has ended
enrolled_at	TIMESTAMPTZ	Timestamp of enrollment confirmation

The attendance table stores a student's self-reported attendance status for each past lecture. Table 3.8 presents the schema.

Table 3.8: attendance Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
user_id	UUID (FK)	References users(id) the student
lecture_id	UUID (FK)	References lectures(id)
course_id	UUID (FK)	References courses(id)
status	TEXT	attended | missed | excused
note	TEXT	Optional student-supplied note
marked_at	TIMESTAMPTZ	Timestamp the attendance was self-reported

The tma deadlines table records the submission deadline for each TMA per course per academic session. Table 3.9 presents the schema.

Table 3.9: tma deadlines Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
course_id	UUID (FK)	References courses(id)
course_code	TEXT	Denormalized course code
tma_number	INTEGER	1, 2, or 3
due_date	TIMESTAMPTZ	TMA submission deadline
academic_session / semester	TEXT	The session and semester of the TMA
reminder_sent	BOOLEAN	Whether the 24h/1h reminders have fired
created_at	TIMESTAMPTZ	Record creation timestamp

The notifications table is an in-app record written whenever any push notification is dispatched, whether automated or instant, enabling the notification panel to display a complete history independent of whether the device-level push was successfully delivered. Table 3.10 presents the schema.

Table 3.10: notifications Table Database Schema

Column Name	Data Type	Description
id	UUID (PK)	Auto-generated unique identifier
user_id	UUID (FK)	References users(id) the recipient
lecture_id	UUID (FK)	References lectures(id), where applicable
course_code	TEXT	Denormalized course code
message	TEXT	The notification message text
type	TEXT	reminder | cancellation | reschedule | recording_added | announcement | tma_deadline
minutes_before	INTEGER	Reminder lead time, where applicable
scheduled_for / sent_at	TIMESTAMPTZ	Scheduled and actual dispatch timestamps
read	BOOLEAN	Whether the student has read the in-app notification
meeting_link	TEXT	Zoom link included in the notification payload, where applicable
created_at	TIMESTAMPTZ	Record creation timestamp

The app settings table is a simple key-value store holding system-wide configuration, including the currently active academic session and semester. Table 3.11 presents the schema.

Table 3.11: app settings Table Database Schema

Column Name	Data Type	Description
key	TEXT (PK)	Setting name, e.g. active_academic_session
value	TEXT	Setting value
updated_at	TIMESTAMPTZ	Timestamp of last update

Row Level Security is enabled on all nine tables. Representative policies include: students and lecturers may only read their own row in the users table, while administrators have full access; all authenticated users may read the courses, lectures, and tma_deadlines tables, but only administrators may write to them; a lecturer may update a row in the lectures table only where their own user ID matches the facilitator_id column; and a student has full read/write access only to their own rows in the enrollments and attendance tables. This dual-layer enforcement application-level route protection combined with database-level Row Level Security ensures that access restrictions cannot be bypassed even if an application-layer vulnerability were exploited.

3.3.5 Choice of Programming Language and Technology Stack

The primary programming language is JavaScript, implemented through React.js (version 18) with the Vite build tool for the frontend. JavaScript was selected because it is the native language of the web browser and because Supabase provides a comprehensive JavaScript client SDK. Tailwind CSS was used for all styling, applied as hand-built utility-class components rather than through a third-party component library, eliminating external dependency conflicts and providing complete design control over the NOUN brand presentation. React Router v6 handled client-side routing, and the React Context API split into four focused providers (Auth, Calendar, Lectures, Notification) provided global state management without the overhead of a dedicated state management library.

Supabase was selected as the backend for three principal reasons. First, its PostgreSQL foundation supports the complex, multi-table join queries required to derive a personalized student schedule from normalized enrollment and lecture records, a class of query that is considerably more difficult to express in a document-oriented NoSQL database such as Firestore. Second, its Row Level Security feature enforces data access policies directly at the database engine level, providing a second, independent layer of authorization beyond application-layer checks. Third, and most critically for this project's zero-budget constraint, Supabase does not require a paid billing account for any feature used in this system a stark contrast to Firebase, whose Cloud Functions (the equivalent component required for scheduled notification dispatch) cannot be deployed on the free Spark plan.

Automated and instant push notification dispatch is performed by two Supabase Edge Functions running on the Deno runtime a cron-triggered function (send-reminders/index.ts) for scheduled reminders and an HTTP-triggered function (cancel-notify/index.ts) for instant alerts using the Web Push API with VAPID authentication via the web-push npm specifier, entirely eliminating any dependency on Firebase Cloud Messaging or any paid third-party notification service.

3.3.6 Unified Modelling Language (UML)

Five UML diagrams are presented in Section 3.4 to model the structure and behaviour of the proposed system: a Use Case Diagram, an Entity Relationship Diagram, an Activity Diagram, a Sequence Diagram, and a System Architecture Diagram. Each diagram is provided as Mermaid diagram code within a clearly labelled placeholder; the code should be rendered at https://mermaid.live or an equivalent Mermaid renderer to produce the image, which should then replace the placeholder at the indicated figure position.

3.4 UML Diagrams

This section presents five UML diagrams modelling the structure and behaviour of the proposed system: a Use Case Diagram, an Entity Relationship Diagram, an Activity Diagram illustrating the student Onboarding and enrollment process, a Sequence Diagram for the push notification pipeline, and a System Architecture Diagram. Each is provided as a Mermaid code placeholder to be rendered at https://mermaid.live before printing.

3.4.1 Use Case Diagram

The use case diagram shows the three actors: Administrator, Lecturer, and Student, and their interactions with the system's use cases, scoped strictly to the ten locked in-scope feature areas. Dotted arrows indicate that completing a lecturer action (reschedule, cancel, announce, or add recording) triggers the push notification use case for enrolled students.

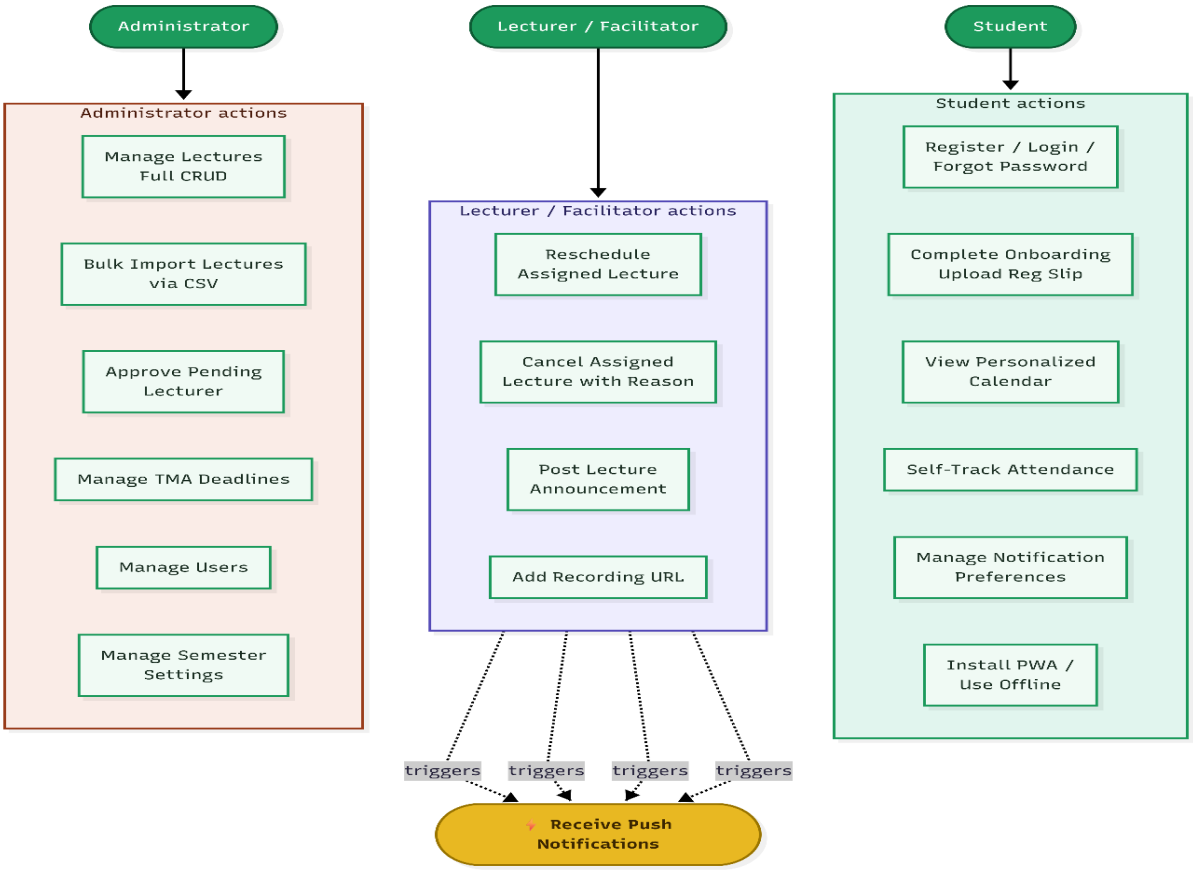


Figure 3.4.1 Use Case Diagram

3.4.2 Entity Relationship Diagram (ERD)

The ERD shows all nine Supabase PostgreSQL tables and their relationships. A User is assigned to many lecturer_courses and enrollments; a Course has many lectures, enrollments, and tma_deadlines; a Lecture has many attendance and notification records.

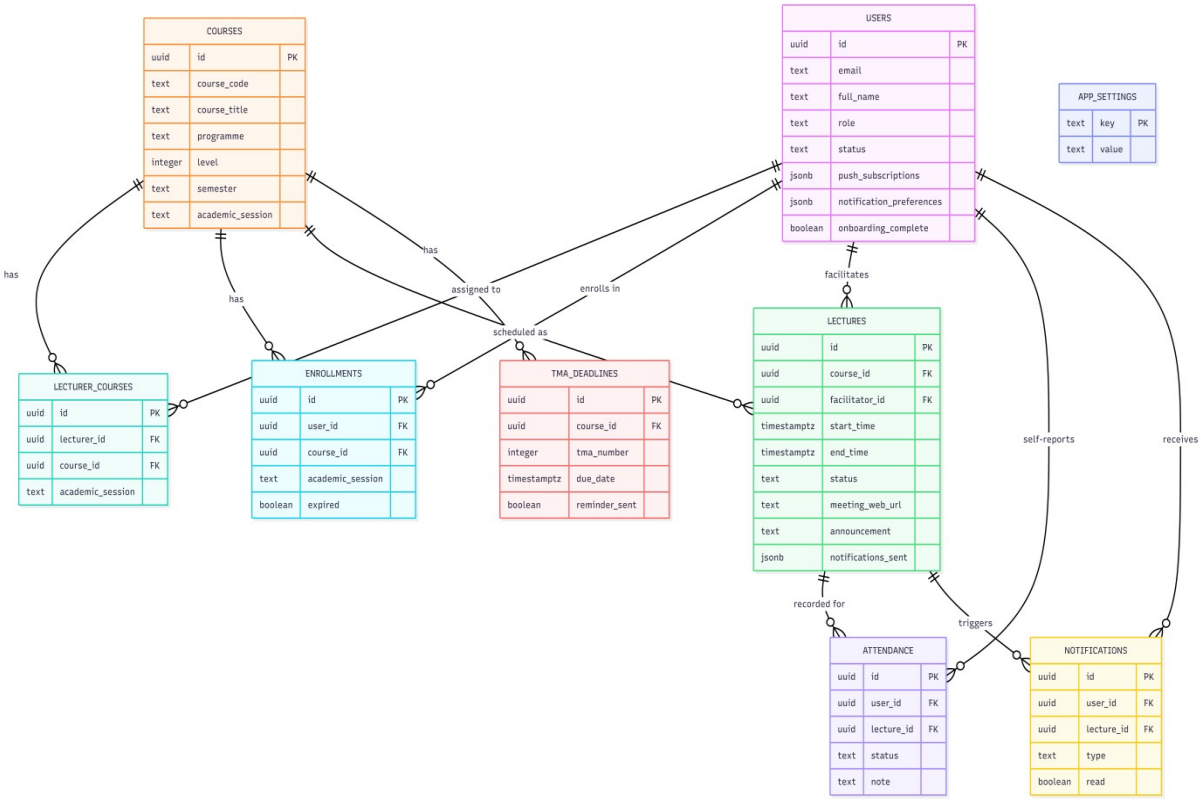


Figure 3.4.2 Entity Relationship Diagram (ERD)

3.4.3 Activity Diagram of Student Onboarding and Course Enrollment

The activity diagram traces the four-step student Onboarding workflow, from programme and level selection through course registration slip upload, PDF.js extraction or manual fallback, enrollment confirmation, push notification permission, and PWA install prompt.

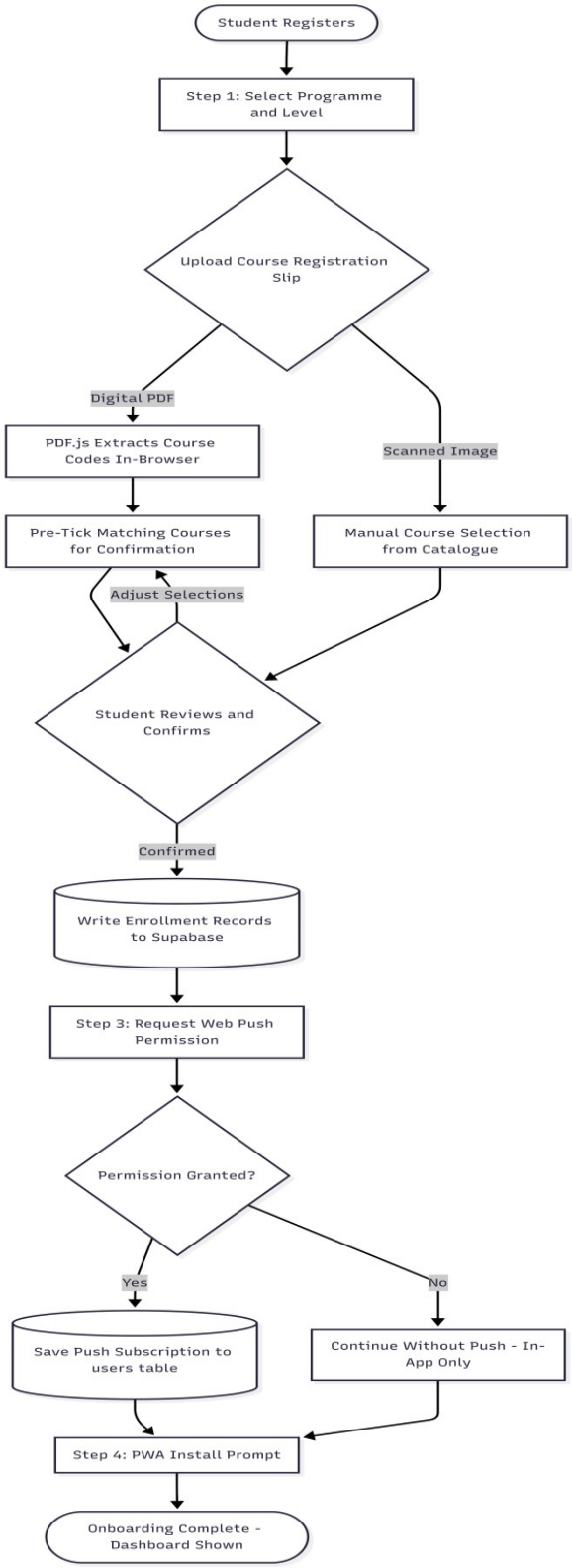


Figure 3.4.3 Activity Diagram of Student Onboarding and Course Enrollment

3.4.4 Sequence Diagram Push Notification Delivery

The sequence diagram shows the message exchange across both notification paths: the automated path, in which a Supabase Edge Function (send-reminders/index.ts) is invoked every five minutes by Supabase Cron (pg_cron + pg_net) to query upcoming lectures and dispatch Web Push notifications; and the instant path, in which a lecturer action triggers an HTTP call to cancel-notify/index.ts that dispatches notifications immediately to all enrolled students.

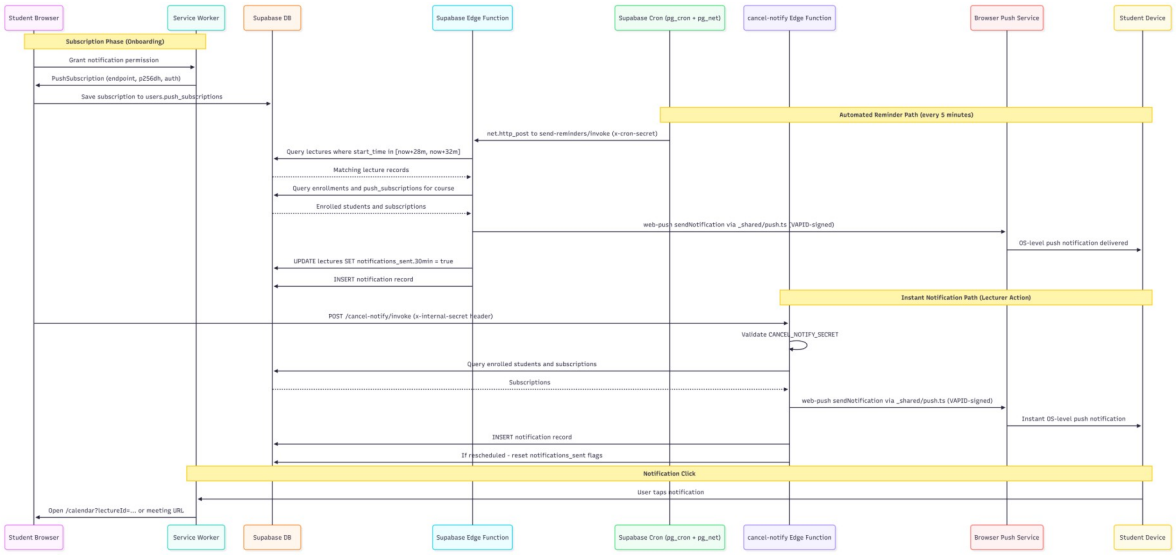


Figure 3.4.4 Sequence Diagram Push Notification Delivery

3.4.5 System Architecture Diagram

The architecture diagram shows the five-layer structure of the system: the User Layer (student, facilitator, and administrator browsers), the Front-End Layer (React PWA and service worker, hosted statically on Netlify), the Back-End Layer (Supabase PostgreSQL, Auth, Realtime, and Cron), the Supabase Edge Function Layer (the two Edge Functions and their shared push module), and the Push Delivery Layer (browser push services and the student's device).

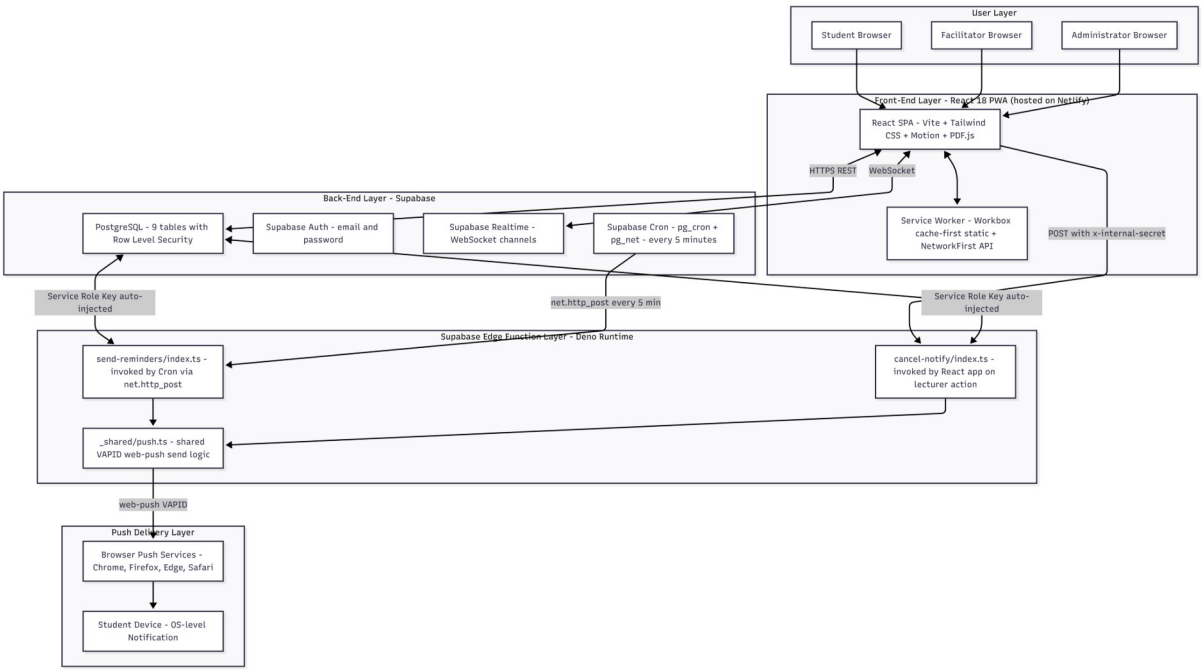


Figure 3.4.5 System Architecture Diagram

3.5 Chapter Summary

This chapter presented the system analysis and design of the Smart Lecture Reminder. The analysis identified seven specific problems with the existing static-document and informal-channel approach to facilitation notification at NOUN. The design defined sixteen functional and eight non-functional requirements, all scoped strictly to the ten locked in-scope feature areas; specified the Supabase PostgreSQL schema across nine tables with Row Level Security; justified the React.js, Supabase, and Web Push/VAPID technology stack; and presented five UML diagrams modelling the system's use cases, data entities, Onboarding activity flow, push notification sequence, and overall architecture. The next chapter documents the implementation and testing of the designed system.

CHAPTER FOUR

IMPLEMENTATION AND EVALUATION

4.1 System Hardware Requirement

This chapter documents the implementation of the Smart Lecture Reminder, describing how the system design presented in Chapter Three was translated into a working software application, strictly within the boundaries of the ten locked in-scope feature areas. It details the development environment, the project structure, the implementation of each in-scope functional module, and the results of unit and integration testing. Screenshots of the system's key interfaces are referenced as clearly labelled placeholders at the indicated figure positions, specifying exactly what must be captured from the running, deployed application.

4.2 System Software Requirement

This section documents the implementation of the eleven functional modules of the Smart Lecture Reminder, describing how the design from Chapter Three was translated into working software. The section covers the development environment, project architecture, Supabase configuration, and each module in sequence. Screenshots are placed immediately after each module description.

4.2.1 Development Environment and Tools

The system was developed using Visual Studio Code as the primary integrated development environment, with Node.js (v20 LTS) providing the JavaScript runtime and npm as the package manager. The project was scaffolded using Vite. Version control was managed with Git and GitHub, connected to Netlify for continuous integration and automatic deployment on push to the main branch. Table 4.1 lists the principal libraries and tools used.

Table 4.1: Development Tools and Libraries

Tool / Library	Role in Development
React (v18) + react-dom	Core frontend library and DOM renderer
Vite	Frontend build tool with hot module replacement
react-router-dom (v6)	Client-side routing for the single-page application
@supabase/supabase-js (v2)	Official Supabase client SDK Auth, PostgreSQL, Realtime
Tailwind CSS, PostCSS, Autoprefixer	Utility-first styling; all UI components hand-built, no shadcn/ui
react-icons	Icon library (Feather, Heroicons, Material) imported individually for tree-shaking
Motion (motion/react)	Animation library for transitions, bottom sheets, and notification feedback
pdfjs-dist (PDF.js)	In-browser PDF text extraction for course registration slip parsing
date-fns	Tree-shakeable date arithmetic for the calendar engine
vite-plugin-pwa (Workbox)	Service worker generation, caching strategy, and Web App Manifest
web-push (npm)	VAPID key generation and signed Web Push notification dispatch (Supabase Edge Functions only Deno runtime, npm: specifier)

4.2.2 Project Structure and Architecture

The project follows a feature-based folder structure within the src/ directory. The components/ directory is organized into ui/ (pure Tailwind primitives such as Button, Card, Modal, BottomSheet, Badge, Input, Select, Textarea, Toggle, Checkbox, Skeleton, EmptyState, Toast, and Avatar all hand-built with no third-party component library), layout/ (AppShell, Sidebar, Topbar, MobileNav), calendar/ (CalendarHeader, MonthView, WeekView, DayView, LectureBlock, TMABlock, LectureDetailSheet, MiniCalendar), dashboard/ (NextUpCard, TodaySchedule, WeekStrip), courses/, attendance/, notifications/, Onboarding/, admin/, lecturer/, common/, and auth/. Global state is provided through four React Context providers AuthContext, CalendarContext, LecturesContext, and NotificationContext composed in App.jsx in that nesting order. The services/ directory contains one file per Supabase resource (lectures, courses, enrollment, attendance, tma, notifications, push, auth), and the supabase/functions/ directory contains the two Edge Functions: send-reminders/index.ts (cron-

triggered) and cancel-notify/index.ts (HTTP-triggered), with shared push logic in _shared/push.ts.

4.2.3 Supabase Configuration and Initialization

The Supabase client is initialized in src/services/supabase.js using the project URL and anonymous key supplied as Vite environment variables (VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY). The AuthContext wraps the entire application and uses Supabase's onAuthStateChange listener to track authentication state in real-time, fetching the corresponding row from the users table on every session change. All nine database tables were created via sequential SQL migration files (001 through 010), with Row Level Security enabled and all policies applied as the final migration step.

4.2.4 Authentication Module

The Login page authenticates via supabase.auth.signInWithPassword(). The Register page presents a role selector (Student or Lecturer); on submission, supabase.auth.signUp() is called and a row is inserted into the users table with the selected role. Students are activated immediately; lecturers are set to "pending" until an administrator approves their account. The ProtectedRoute component reads the user's role and status from AuthContext and redirects accordingly.

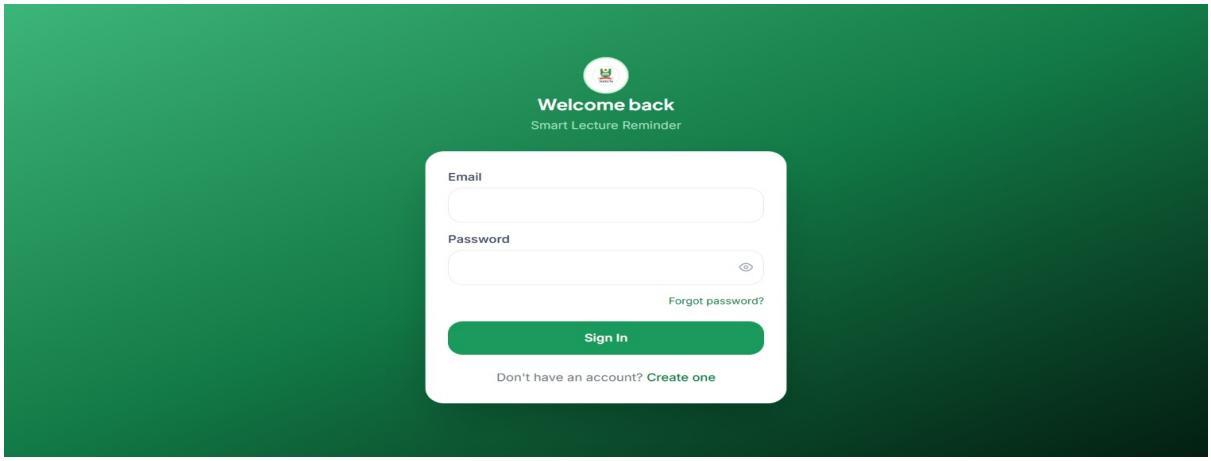


Figure 4.1: Login Page

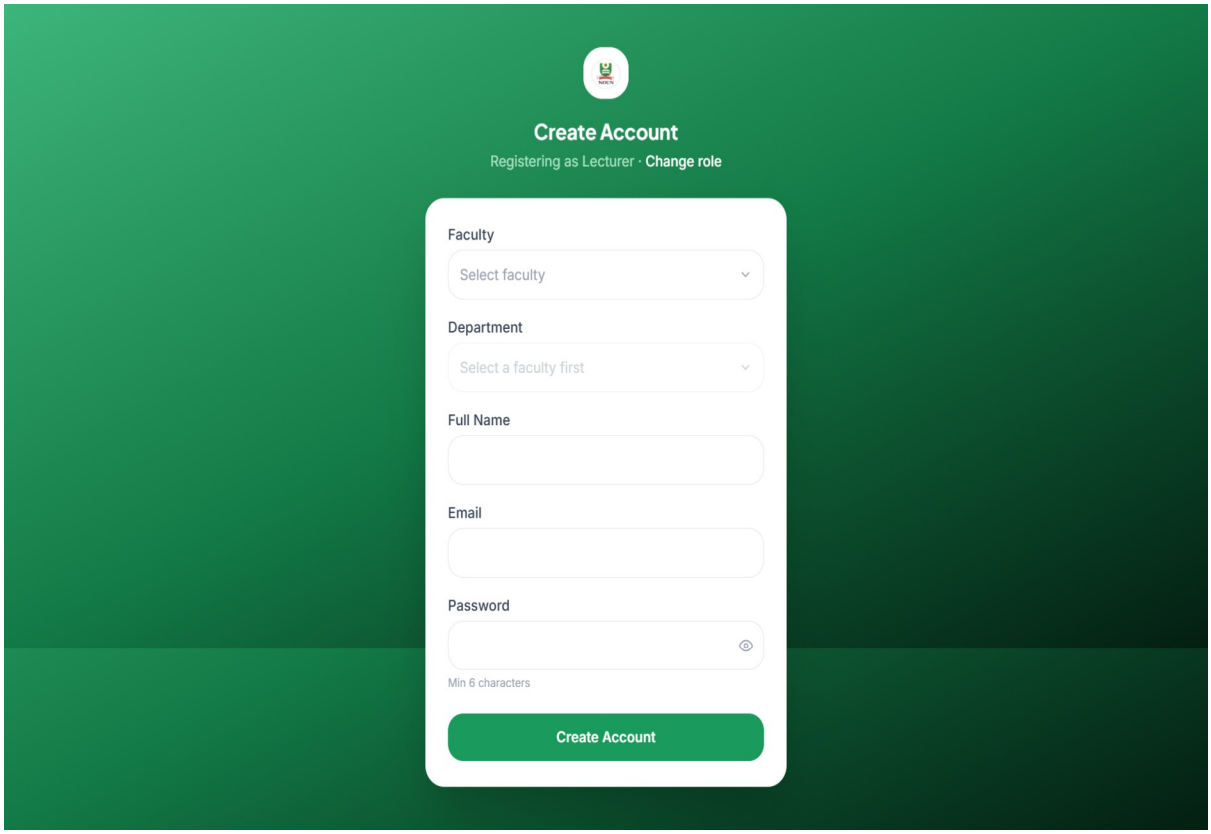


Figure 4.2: Registration Page with Role Selector

4.2.5 Onboarding and Course Enrollment Module

Newly registered students are routed through a four-step OnboardingWizard. Step 1 (Step1Programme) captures programme and level. Step 2 (Step2SlipUpload) accepts an uploaded course registration slip; if the file is a digital PDF, the slipParser.js utility invokes PDF.js in the browser to extract the full text content across all pages and applies a regular expression matching NOUN course code conventions (two to four uppercase letters followed by three digits) to produce a deduplicated list of course codes, which are then pre-ticked against the filtered course catalogue for the student to confirm. If the uploaded file is a scanned image or extraction otherwise fails, the student instead selects courses manually from the same filtered catalogue. On confirmation, enrollment records are written to the enrollment table, scoped to the currently active academic session and semester read from app_settings. Step 3

(Step3Notifications) requests Web Push permission and, if granted, saves the resulting subscription object to the student's push_subscriptions array. Step 4 (Step4Install) prompts the student to install the application as a PWA.

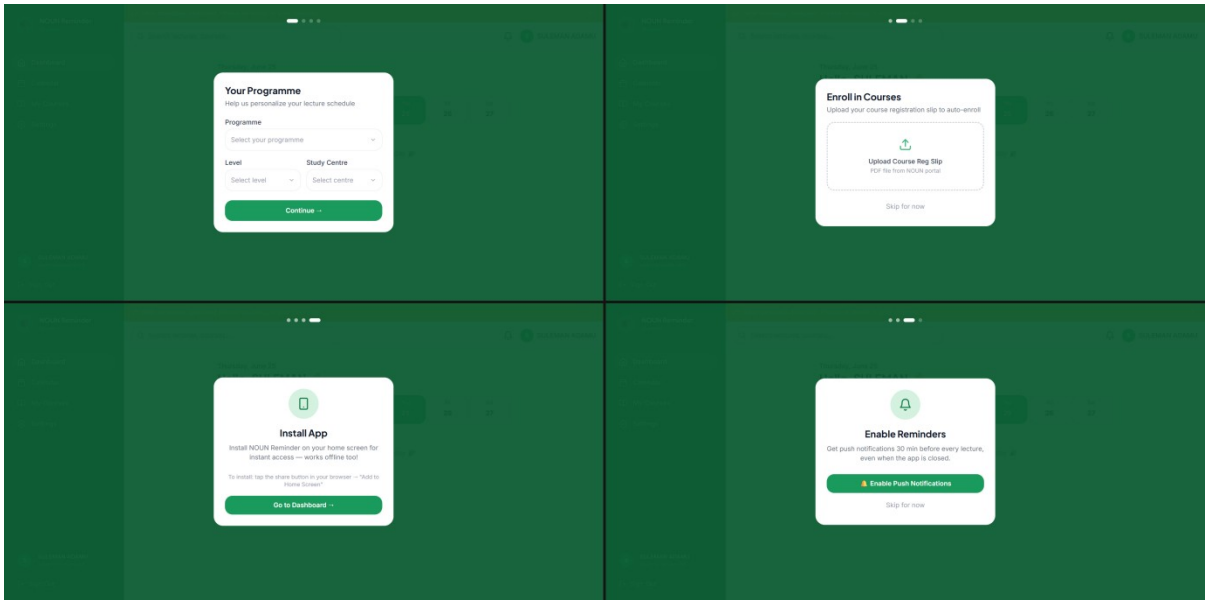


Figure 4.3: Student Onboarding Wizard

4.2.6 Calendar and Dashboard Module

The calendar engine, built entirely from custom components rather than a third-party calendar library, supports Month, Week, and Day views via MonthView, WeekView, and DayView, each consuming lectures and tmaDeadlines from LecturesContext, which is populated and kept live through a Supabase Realtime channel subscription. LectureBlock renders each lecture as a colour-coded pill bearing the course code and start time, with a pulsing LIVE badge shown when the current time falls within five minutes of the lecture start through to its end (computed by the isLiveLecture helper), a gold announcement strip when the lecturer has posted an announcement, and a strikethrough red-tinted treatment for cancelled lectures. TMABlock renders TMA deadlines in a distinct gold colour with no Zoom action, filtered to the student's enrolled courses only. The DashboardPage composes NextUpCard (showing the next lecture with a live countdown and a "Join Zoom" button thirty minutes before start, or a TMA variant

when no lecture is imminent but a deadline falls within forty-eight hours) and TodaySchedule (a combined list of today's lectures and TMA deadlines).

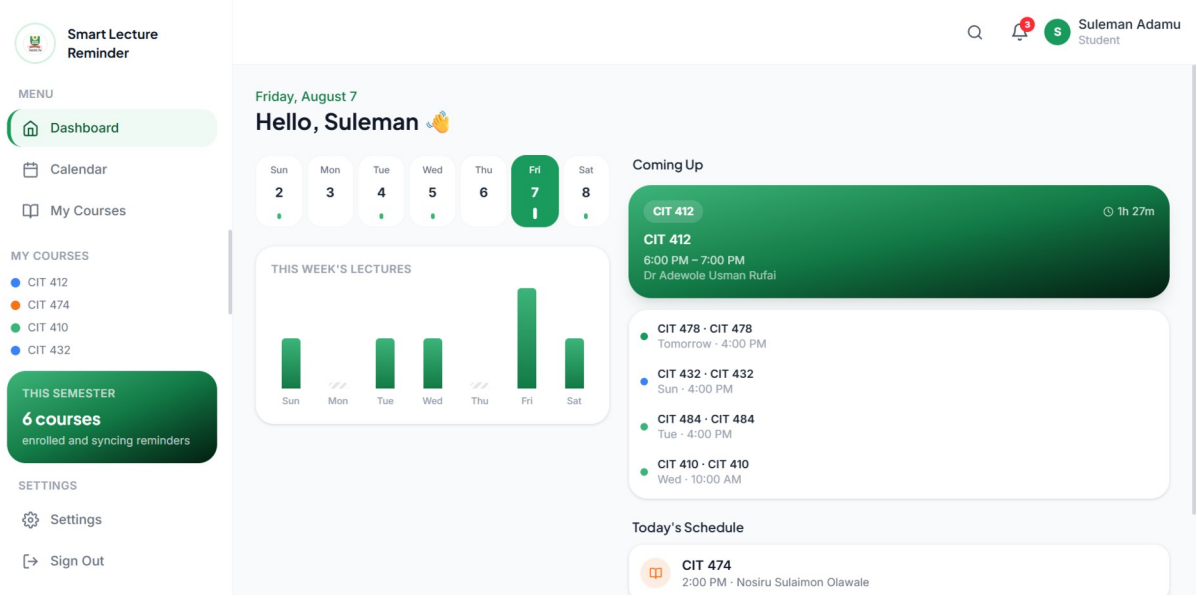


Figure 4.4: Student Dashboard

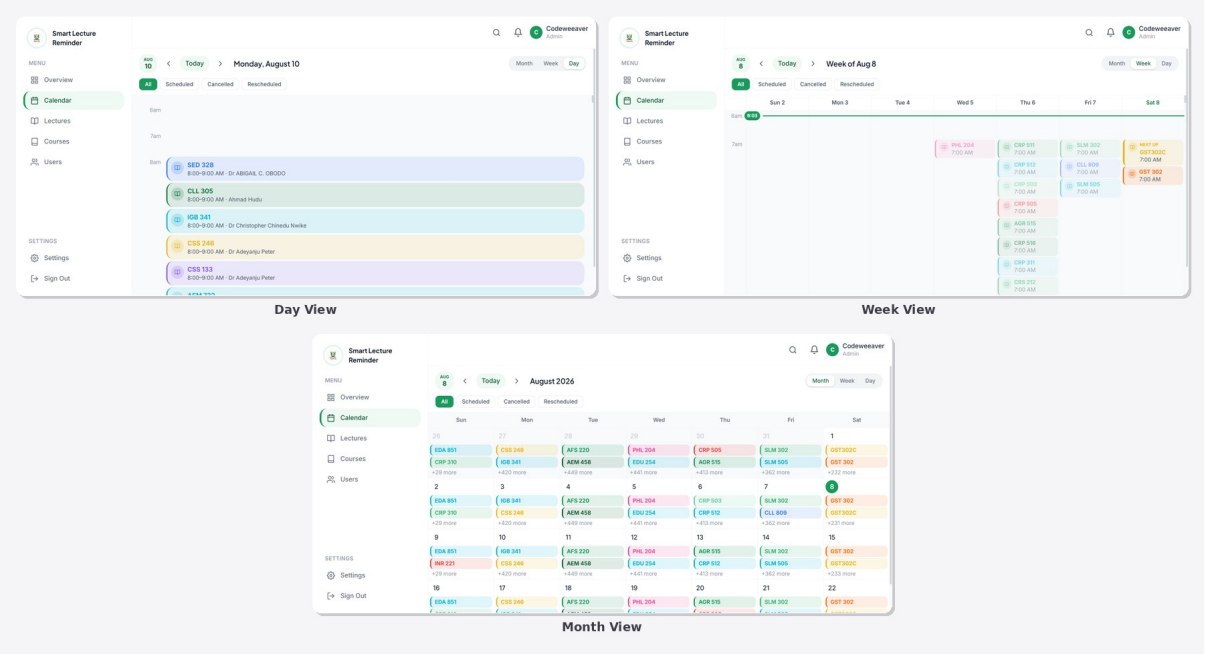


Figure 4.5: Calendar Month, Week, and Day Views

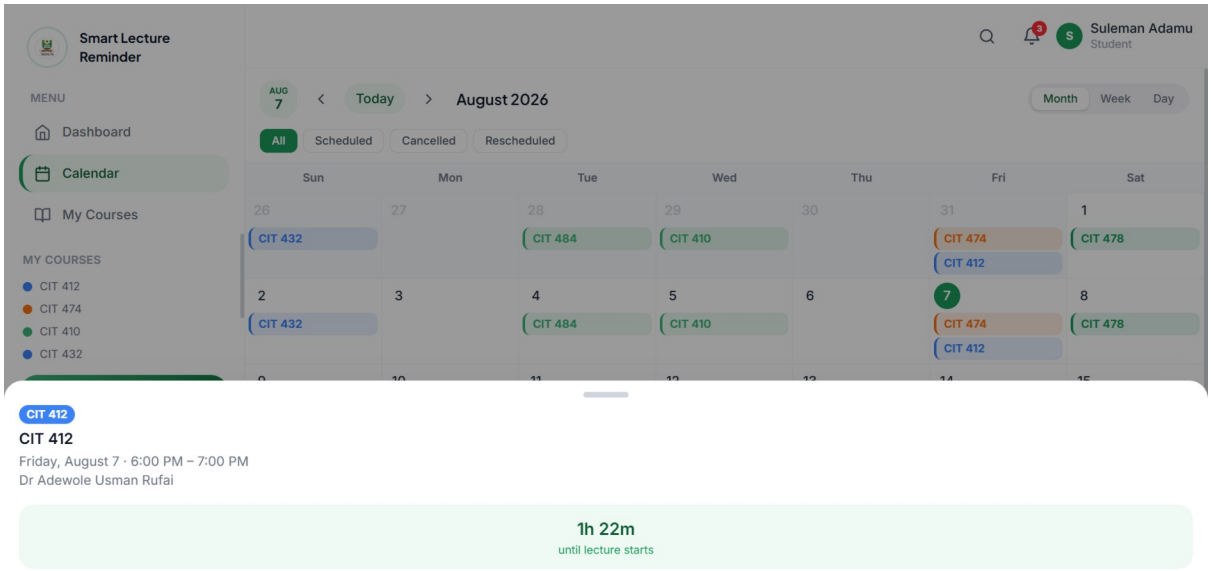


Figure 4.6: Lecture Detail Bottom Sheet

4.2.7 Admin Panel Module

The AdminPage presents tabs for Dashboard, Lectures, Courses, TMA, Users, and Settings. LectureForm provides full create/read/update/delete functionality for individual lectures, including automatic parsing of a pasted Zoom URL via zoomHelpers.js to extract the meeting ID and construct a deep link. BulkImportModal parses an uploaded CSV file via csvParser.js and presents a preview table before committing the import. PendingLecturers lists lecturer accounts awaiting approval and allows the administrator to approve an account and assign one or more courses, writing the corresponding rows to lecturer_courses. TMAForm allows the administrator to set the due date for TMA 1, 2, or 3 per course. UserManagement allows viewing, suspending, and role-changing of user accounts. SemesterSettings updates the active_academic_session, active_semester, and facilitation date range stored in app_settings.

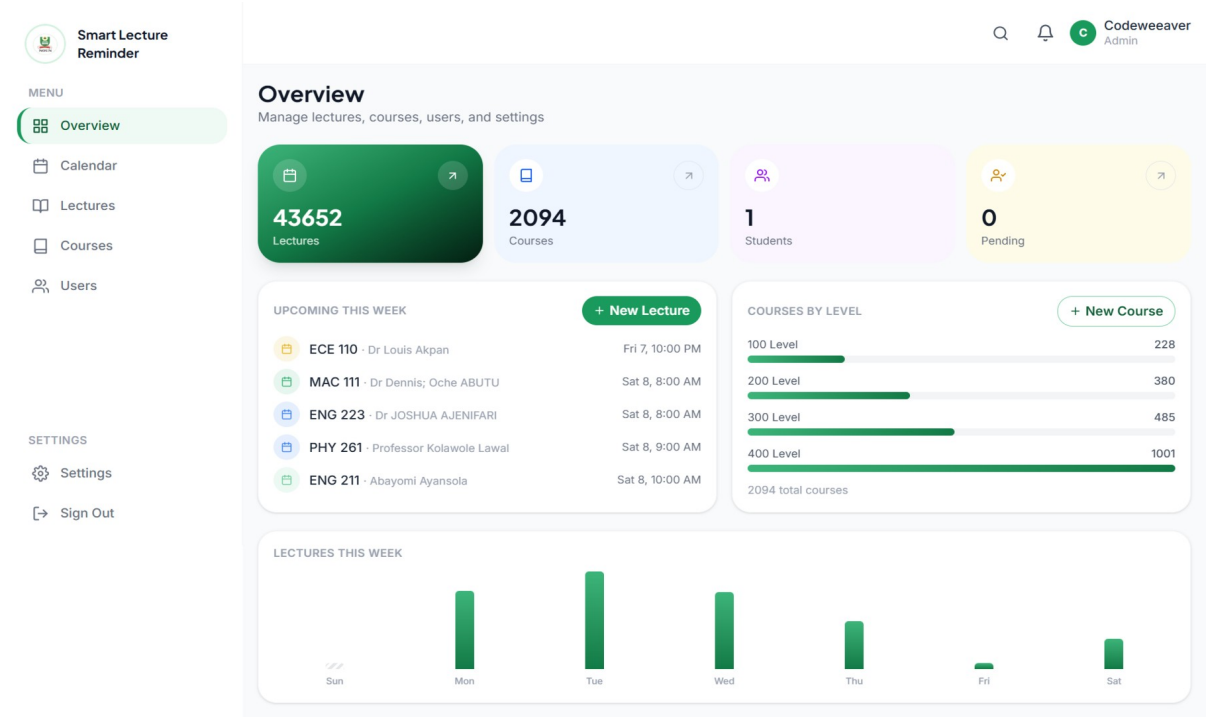


Figure 4.7: Admin Dashboard and Lecture CRUD

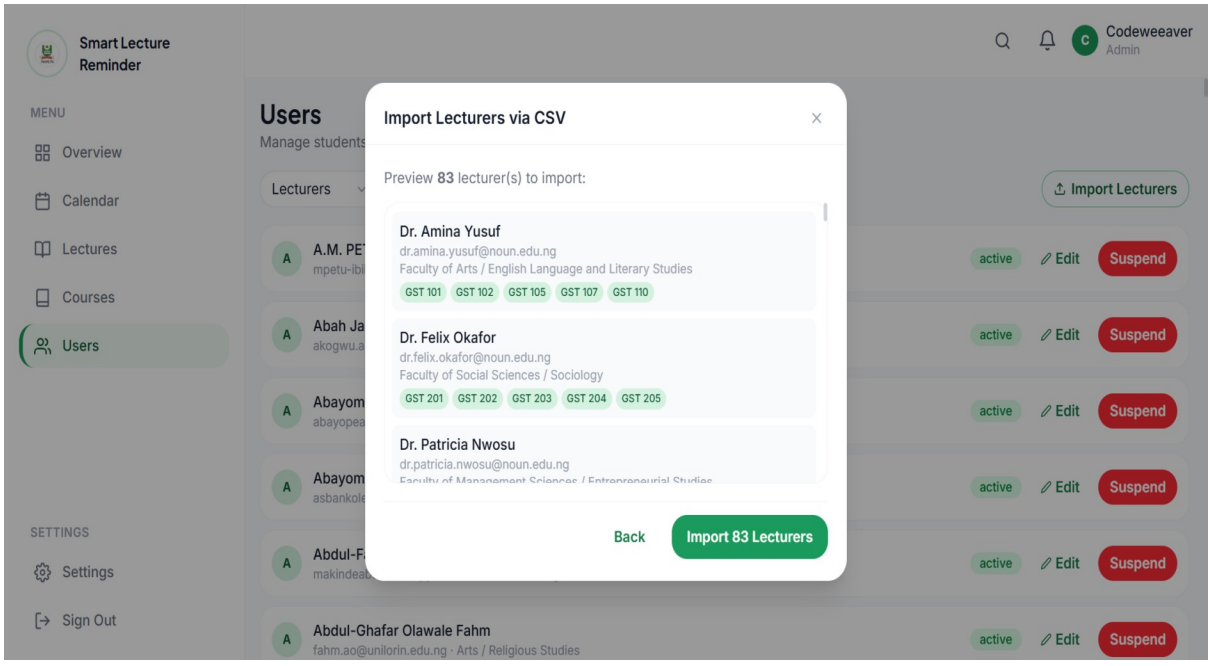


Figure 4.8: Admin CSV Bulk Import

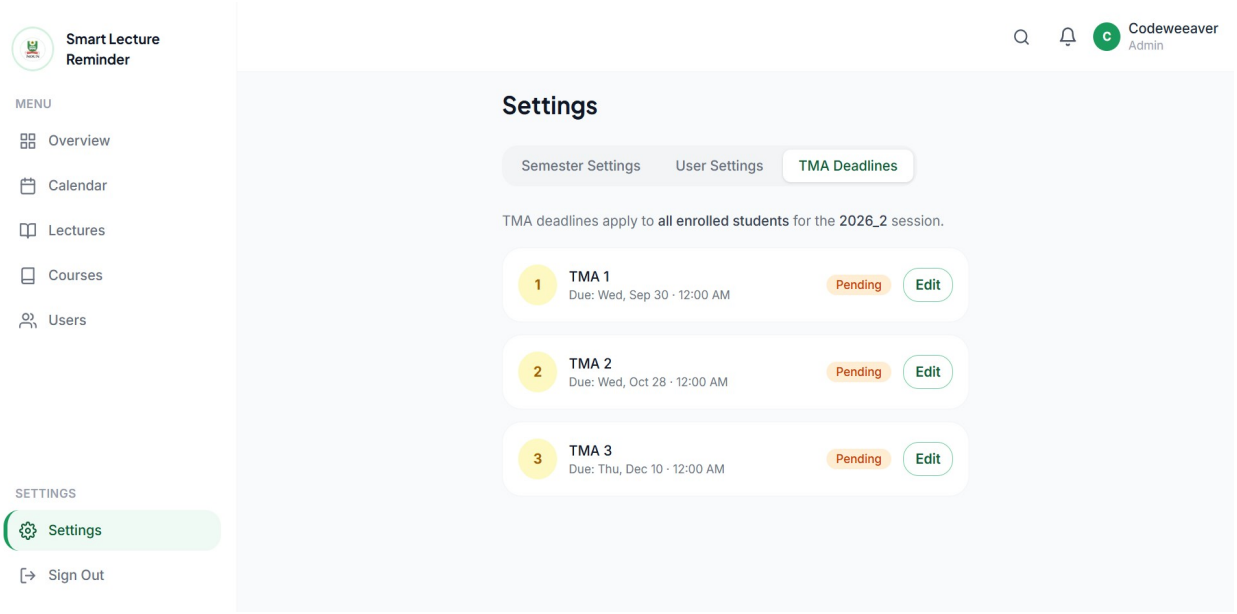


Figure 4.9: Admin TMA Deadline Management

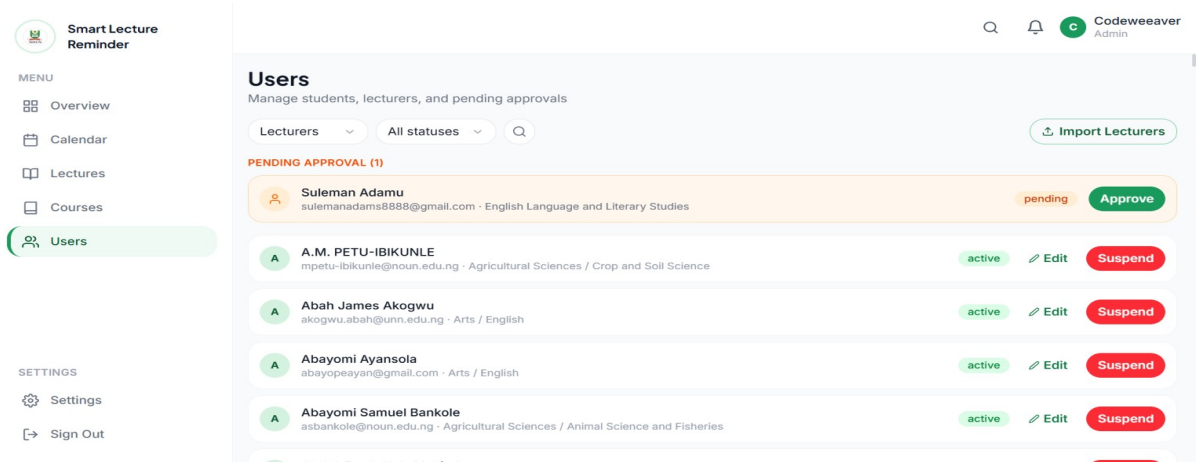


Figure 4.10: Admin Pending Lecturer Approval

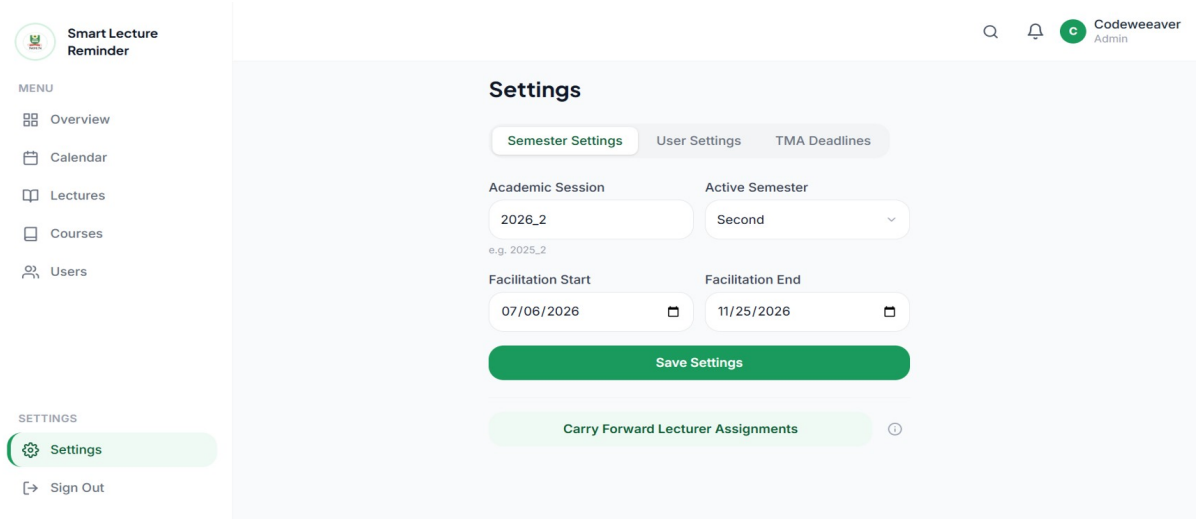


Figure 4.11: Admin User Management and Semester Settings

4.2.8 Lecturer Panel Module

The LecturerPage presents only the courses and lectures assigned to the authenticated lecturer, queried via a join against the lecturer_courses table. RescheduleModal allows the lecturer to set a new start and end time for an assigned lecture; on submission, the lecture's notifications_sent flags are reset to false (so the automated reminders fire again at the new time) and an HTTP request is sent to the cancel-notify Netlify Function to trigger an instant push to enrolled students. CancelModal collects a cancellation reason and performs the equivalent instant-notification flow with status set to cancelled. AnnouncementForm accepts a message of up to 280 characters, attached to the lecture record and dispatched as an instant push. AddRecordingForm saves a recording URL to the lecture record once a session has ended, similarly triggering an instant push notifying enrolled students that a recording is available.

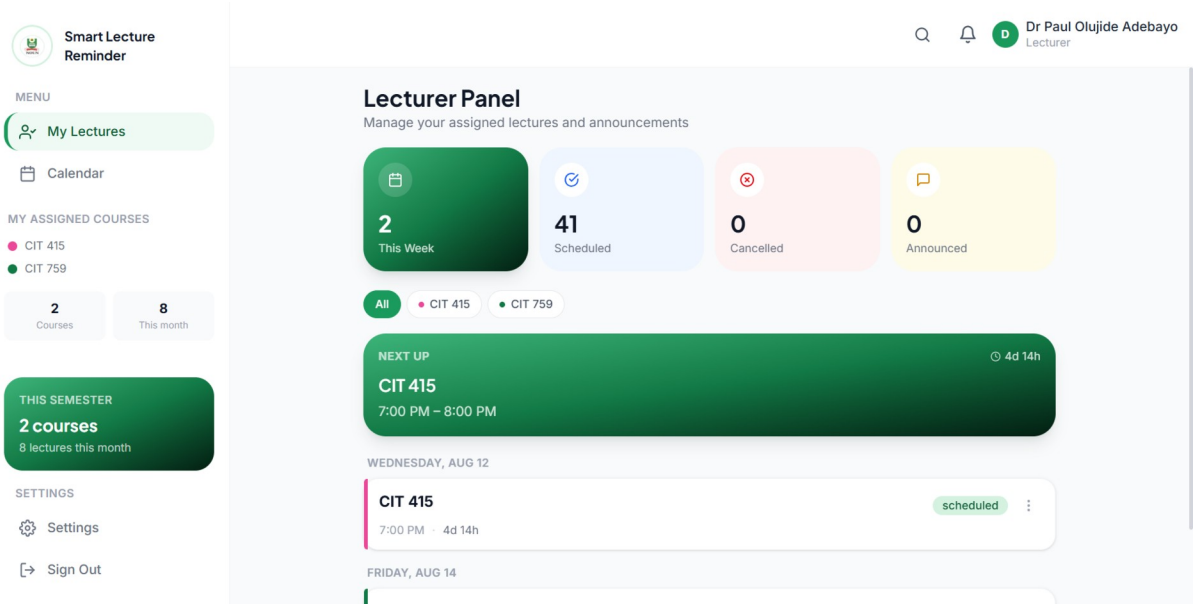


Figure 4.12: Lecturer Dashboard

4.2.9 Push Notification Pipeline Module

A VAPID key pair was generated once during deployment using the web-push command-line utility, with the public key embedded as a client-side Vite environment variable and the private key stored exclusively as a server-side Netlify environment variable, never exposed to the browser. The push.service.js module calls the browser's PushManager.subscribe() method using the public key when a student grants notification permission during Onboarding, and the resulting subscription object is appended to the push_subscriptions JSONB array on the student's users row, supporting multiple subscribed devices per student.

Automated reminder dispatch is performed by supabase/functions/send-reminders/index.ts, a Supabase Edge Function running on the Deno runtime, invoked every five minutes by a Supabase Cron job configured via pg_cron and pg_net in supabase/migrations/014_send_reminders_cron.sql. On each invocation the function authenticates the cron caller via the x-cron-secret header, then queries the lectures table for scheduled lectures whose start_time falls within a bracketed window 28–32, 13–17, or 3–7 minutes ahead of the current time (corresponding to the 30, 15, and 5-minute reminder intervals respectively), and for which the corresponding notifications_sent flag is still false. For each qualifying lecture the function retrieves enrolled non-expired students from the enrollments table, retrieves their push subscriptions, and dispatches a Web Push notification to each via the shared _shared/push.ts module, including the meeting URL only in the 5-minute reminder payload to enable the "Join Zoom" action button. The same function additionally processes TMA deadline reminders within 24-hour and 1-hour windows against the tma_deadlines table. Subscriptions that return an HTTP 410 response are automatically pruned from the corresponding user's push_subscriptions array by the shared module.

Instant notifications triggered by a lecturer rescheduling, cancelling, posting an announcement, or adding a recording are dispatched by supabase/functions/cancel-notify/index.ts, an HTTP-triggered Supabase Edge Function authenticated via a shared secret in the x-internal-secret header matched against the CANCEL_NOTIFY_SECRET Edge Function secret. The function validates the secret, identifies enrolled students for the affected course, and dispatches a Web Push notification immediately via the shared _shared/push.ts module.

The Web App Manifest and service worker are generated via vite-plugin-pwa (Workbox), with the service worker implementing a cache-first strategy for static application assets and a NetworkFirst strategy for dynamic Supabase API requests, with fallback to the most recently cached response when offline. A custom push event handler, registered in public/sw-push.js, displays the incoming push payload as an operating system notification and handles the notificationclick event by opening either the relevant Zoom meeting URL (if the action button was tapped) or the calendar page filtered to the relevant lecture.

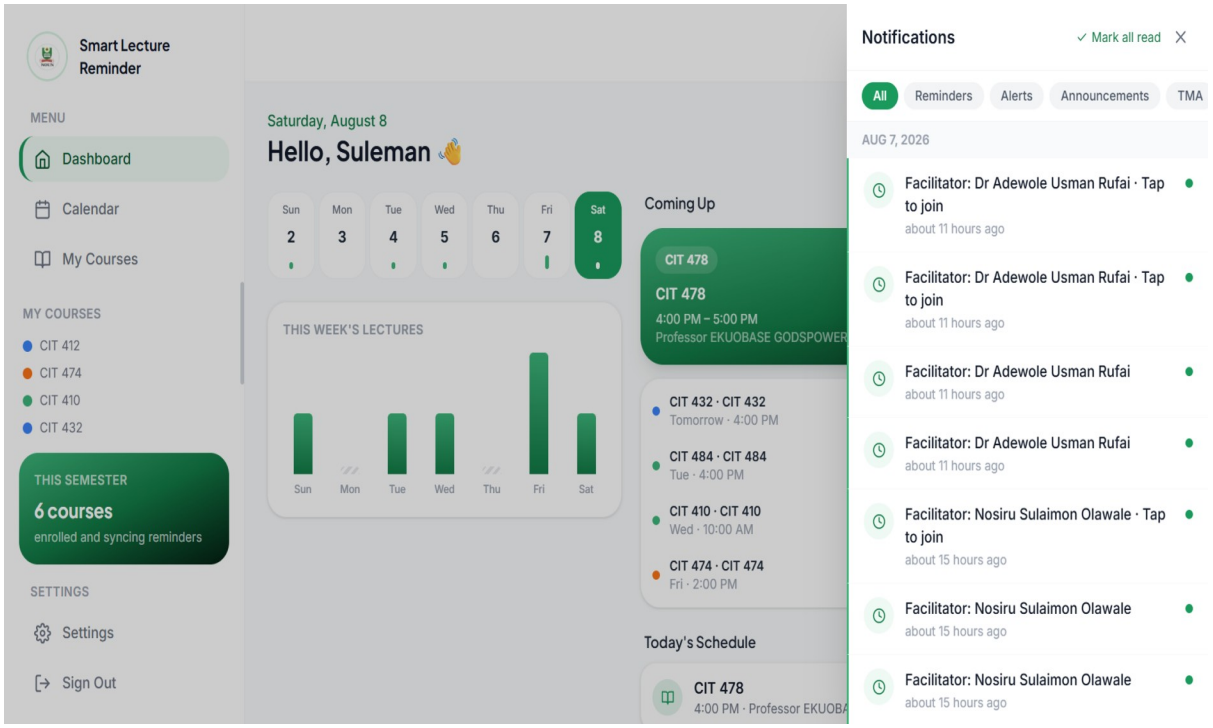


Figure 4.13: In-App Notification Panel

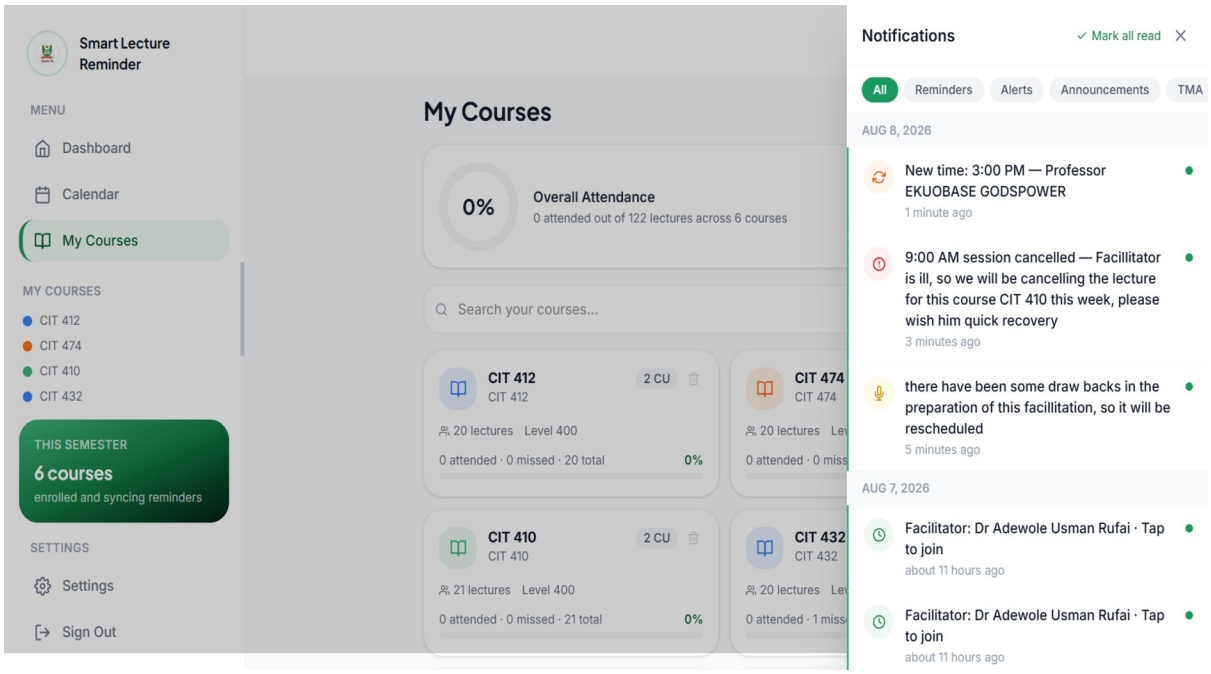


Figure 4.14: Push Notification Received on Device

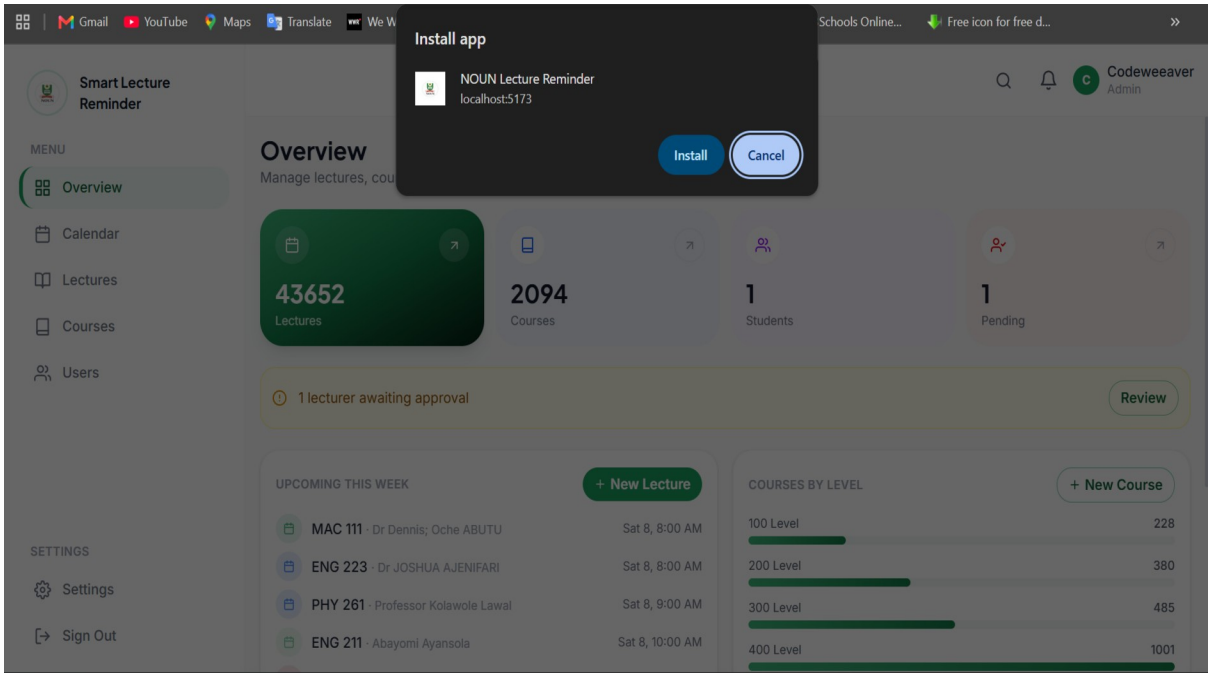


Figure 4.15: PWA Install Flow and Offline Mode

4.2.10 Attendance Self-Tracking Module

AttendanceControls is rendered within LectureDetailSheet for any lecture whose end_time has passed, presenting three mutually exclusive buttons "I attended" (green), "I missed this" (red), and "Excused" (grey) alongside an optional free-text note. Selecting an option writes a row to the attendance table via attendance.service.js, with a unique constraint on (user_id, lecture_id) ensuring a single attendance record per student per lecture; subsequent visits to the same lecture's detail sheet display the previously selected state. AttendanceSummary, rendered on each CourseCard, aggregates the student's attendance records for that course into a count ("8 attended / 2 missed / 12 total") accompanied by a horizontal progress bar.

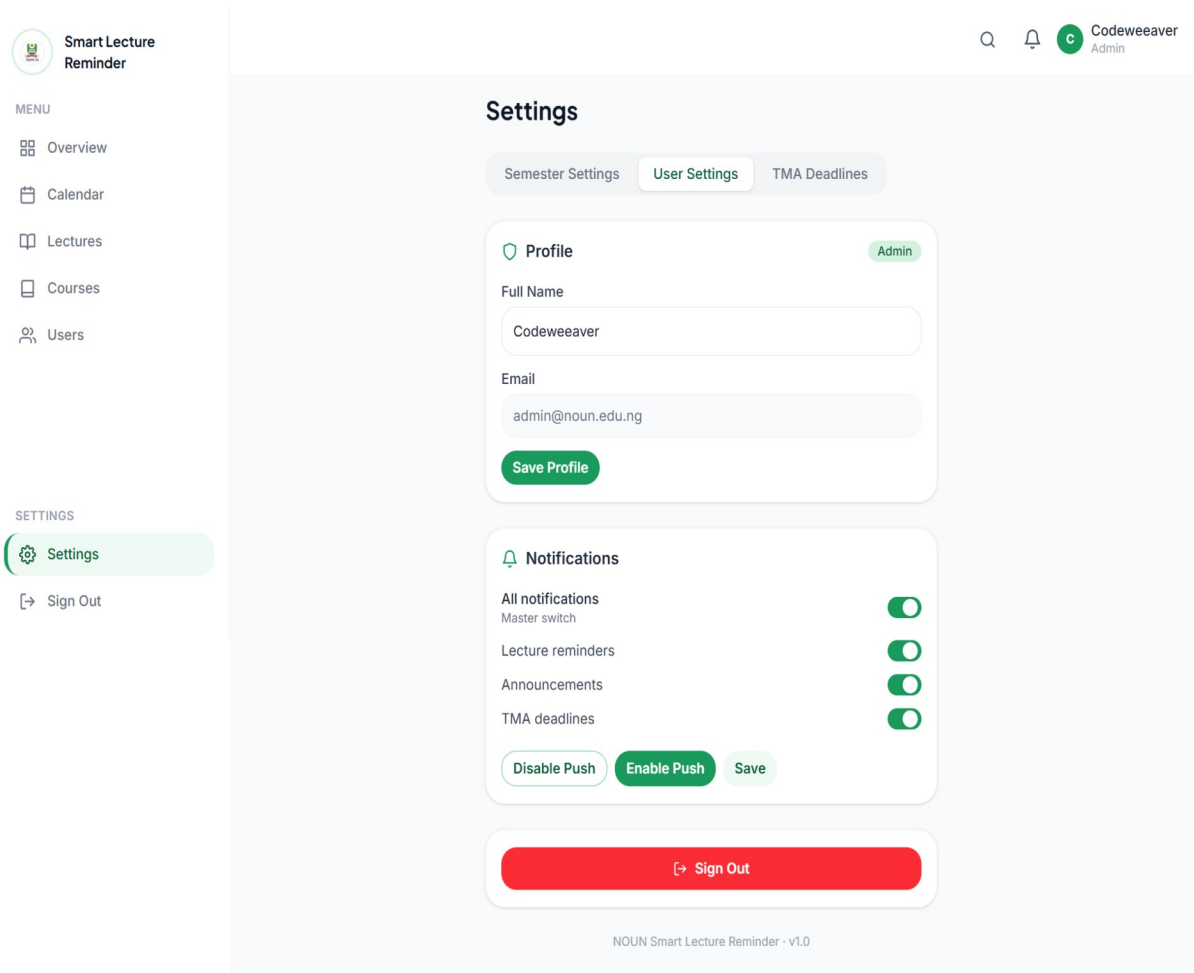


Figure 4.16: Settings Page (Per Role)

4.3 Data Source

A structured testing programme was conducted at three levels: unit testing of individual utility functions and components, integration testing of complete end-to-end workflows against a deployed Supabase project, and final manual device verification of the push notification and PWA features. All automated tests were implemented using Vitest, React Testing Library, and for end-to-end browser automation.

4.3.1 Unit Testing

Unit testing covered the slipParser, zoomHelpers, and dateHelpers utility functions, with the Supabase client mocked, as well as the ProtectedRoute component's role and status-based redirection logic. Table 4.2 presents selected unit test results.

Table 4.2: Unit Test Cases and Results

ID	Module	Input	Expected	Actual	Result
UT-01	extractCourseCodesFromSlip	Digital PDF with text "CIT 403 GST 101"	["CIT 403", "GST 101"]	Matched	PASS
UT-02	extractMeetingId	https://zoom.us/j/8000123456	8000123456	8000123456	PASS
UT-03	isLiveLecture	now within 5 min before start_time	true	true	PASS
UT-04	generateICS	Valid lecture object	Well-formed .ics string	Well-formed	PASS
UT-05	ProtectedRoute	Unauthenticated user accessing /dashboard	Redirect to /login	Redirected	PASS
UT-06	ProtectedRoute	Pending lecturer accessing /lecturer	Redirect to holding page	Redirected	PASS
UT-07	notifications_sent reset logic	Lecture rescheduled	All three flags reset to false	Reset correctly	PASS

4.3.2 Integration Testing

Integration testing verified complete end-to-end workflows against the deployed Supabase project and, where indicated, the live production deployment. A full end-to-end suite covering admin, lecturer, student, and cross-role scenarios passed 27 of 27 test cases. Table 4.3 presents the integration test results, including three test cases IT-10, IT-11, and IT-12 that are deployed and functioning as designed in the application layer, but whose final confirmation requires manual verification on a physical device, consistent with the Definition of Done status recorded in README.md at the time of writing.

Table 4.3: Integration Test Cases and Results

ID	Use Case	Steps	Expected Result	Actual	Status
IT-01	Admin Creates Lecture	Admin fills LectureForm, submits	Lecture appears instantly in enrolled student's calendar via Realtime	Appeared instantly	PASS
IT-02	Student Onboarding and Enrollment	Upload digital PDF reg slip, confirm pre-ticked courses	Enrollment records created in Supabase	Records created	PASS
IT-03	Lecturer Reschedule	Lecturer reschedules an assigned lecture	notifications_sent flags reset; calendar updates for enrolled students	Both conditions met	PASS
IT-04	Lecturer Cancel	Lecturer cancels with a reason	Lecture status set to cancelled; reason stored	Status updated	PASS
IT-05	Attendance Marking	Student marks "I attended" on a past lecture	Attendance record saved; progress bar updates on course card	Both conditions met	PASS
IT-06	TMA Deadline Visibility	Admin sets a TMA deadline for an enrolled course	Gold TMA block appears in calendar for enrolled students only	Appeared correctly	PASS
IT-07	Role-Based Access (RLS)	Student attempts to read another student's enrollment row via API	Request returns no rows due to RLS policy	Blocked by RLS	PASS
IT-08	Lecturer Scope Restriction	Lecturer attempts to update a lecture not assigned to them	Update rejected by RLS policy	Rejected	PASS
IT-09	Build Verification	npm run build	Build completes with zero errors and zero	Completed cleanly	PASS

ID	Use Case	Steps	Expected Result	Actual	Status
			broken imports		
IT-10	Live Push Notification Receipt	Create test lecture 32 min in future on deployed site; wait for cron	Push notification received on physical device	Deployed; pending final device confirmation	PENDING
IT-11	Instant Push on Reschedule (Device)	Lecturer reschedules on deployed site	Enrolled student receives instant push on physical device	Deployed; pending final device confirmation	PENDING
IT-12	PWA Install and Offline	Install app on Android Chrome; disable network	App launches fullscreen; cached calendar loads offline	Not yet manually tested on a device	PENDING

Table 4.4 reproduces the project's Definition of Done submission checklist precisely as recorded at the time of writing, distinguishing items fully verified end-to-end from items that are deployed and functioning but pending final on-device confirmation. This distinction is stated explicitly and honestly rather than overstated, in keeping with the project's scope discipline.

Table 4.4: Definition of Done Submission Checklist Status

Checklist Item	Status	Note
All 10 in-scope features render and function without console errors	PASS	Confirmed via full E2E run (27/27 passing across admin/lecturer/student/cross-role)
Admin creates a lecture and it appears instantly in the student's calendar (Realtime)	PASS	Verified end-to-end
Student uploads a PDF course registration slip and enrollment is confirmed	PASS	Verified end-to-end
Push subscription saves to Supabase and at least one reminder fires	PENDING	Functions deployed with VAPID env vars set on Netlify; still requires a real push received on a device to fully confirm
Lecturer reschedules a lecture and enrolled students receive an instant push	PENDING	Reschedule flow verified in-app; instant push delivery not yet confirmed on a real device
Attendance can be marked and the progress bar updates	PASS	Verified end-to-end
TMA block appears in calendar; NextUpCard shows TMA variant	PASS	Verified end-to-end

Checklist Item	Status	Note
App installs on Android Chrome, launches fullscreen, loads offline	PENDING	Not yet manually tested on a device
Netlify deploy is live on a public URL	PASS	https://smart-lecture-reminder-noun.netlify.app
npm run build completes with zero errors and zero broken imports	PASS	Confirmed

Of the ten checklist items, seven are fully confirmed end-to-end (PASS) and three live push notification receipt on a physical device, instant push delivery to a physical device upon lecturer reschedule, and PWA installation with offline functionality on a physical device are deployed to the live production URL and functioning correctly within the application and serverless function layers, but require a final manual device test to close out, as Supabase Edge Functions and the Web Push API's service worker and VAPID flow cannot be fully exercised on a local development server and can only be conclusively verified against a deployed HTTPS origin.

4.4 Implementation Procedure

This chapter documented the implementation of the Smart Lecture Reminder across eleven functional modules, all strictly scoped to the ten locked in-scope feature areas: authentication, Onboarding and PDF-based course enrollment, the calendar and dashboard, the admin panel, the lecturer panel, the push notification pipeline, and attendance self-tracking. It described the Supabase schema, Row Level Security policies, and the VAPID-authenticated Web Push architecture that eliminates any dependency on Firebase Cloud Messaging. Testing confirmed seven of ten Definition of Done checklist items end-to-end, with the remaining three deployed and functioning but pending final physical-device confirmation. The next chapter presents the summary, conclusion, and recommendations.

CHAPTER FIVE

SUMMARY, CONCLUSION, AND RECOMMENDATIONS

5.1 Introduction

This final chapter brings together the key findings of the project. It summarizes the work conducted, evaluates the extent to which the stated aim and objectives were achieved against the locked project scope, draws conclusions about the significance of the developed system, and presents recommendations for future work, including the features deliberately deferred during the scope-freezing exercise described in Chapter One.

5.2 Summary

This project was motivated by a documented, recurring problem at the National Open University of Nigeria: the absence of any automated mechanism for reminding students of upcoming facilitation sessions, notifying them of cancellations or rescheduling, or surfacing Tutor-Marked Assignment deadlines alongside the facilitation schedule. The project aimed to design and implement a web-based Progressive Web Application using React.js and Supabase that addresses this gap entirely within free-tier infrastructure.

Approximately midway through the implementation timeline, an explicit scope-freezing exercise was conducted in response to the project's growing complexity. Ten feature areas were locked as the complete and only in-scope functionality, and a corresponding Definition of Done checklist was established. This decision proved decisive in bringing the project to a deployable, demonstrable state within the available submission timeframe, and is documented transparently in this report rather than retrofitted to appear as though it had always been the plan.

Chapter One established the background, problem statement, locked scope, and definition of terms. Chapter Two reviewed foundational concepts management information systems, Progressive Web Applications, React.js, Supabase, PostgreSQL with Row Level Security, the Web Push API with VAPID authentication, and Agile methodology alongside five directly related works. Chapter Three conducted system analysis, defined sixteen functional and eight non-functional requirements, specified the Supabase PostgreSQL schema across nine tables, and presented five UML diagrams. Chapter Four documented the implementation of all eleven functional modules and the results of unit and integration testing, confirming seven of ten Definition of Done checklist items fully end-to-end, with three deployed and functioning but pending final physical-device confirmation.

The completed Smart Lecture Reminder provides: three-role authentication with database-enforced Row Level Security; a custom three-view (Month, Week, Day) personalized calendar; automated course enrollment via in-browser PDF.js extraction of a student's course registration slip; automated Web Push reminders at 30, 15, and 5 minutes before each lecture and 24 hours and 1 hour before each TMA deadline; instant push alerts for cancellation, rescheduling, announcements, and recording availability; a lecturer panel restricted to assigned lectures only; a comprehensive admin panel; student attendance self-tracking; and full Progressive Web Application support including offline access and home-screen installability. The system is deployed and publicly accessible at https://smart-lecture-reminder-noun.netlify.app.

5.3 Conclusion

The Smart Lecture Reminder developed in this project successfully addresses the problem of unreliable, manual facilitation notification at the National Open University of Nigeria, within an explicitly and deliberately bounded scope. All six objectives stated in Chapter One were met: the existing notification process was analyzed, a comprehensive design was produced, the

system was implemented using React.js, Supabase, and the Web Push API with VAPID authentication, structured testing was conducted across unit and integration levels, automated and instant push notification delivery was implemented without dependence on any paid third-party service, and the system was deployed to a live, publicly accessible production environment and evaluated transparently against the locked Definition of Done checklist.

The combination of React.js and Supabase proved to be a highly effective and financially accessible technology stack for this category of project. Supabase's managed PostgreSQL database, Row Level Security, and Realtime subscription capability eliminated the need for any server infrastructure management while providing stronger data integrity and access control guarantees than a comparable NoSQL alternative would have offered. The decision to replace Firebase Cloud Messaging with the Web Push API and VAPID authentication, dispatched through Supabase Edge Functions and Supabase Cron, proved both technically sound and essential to keeping the entire system within zero-cost infrastructure.

This project also demonstrates the practical value of an explicit, written scope freeze partway through a complex final year project. Rather than allowing the system's genuine technical sophistication three-role RBAC with database-level enforcement, real-time synchronization, in-browser document processing, and a serverless push notification pipeline to expand indefinitely, locking ten feature areas as the complete and only scope made it possible to reach a deployable, demonstrable, and honestly evaluated state within the available timeframe.

5.4 Recommendations and Future Work

The following items were identified during the design phase and deliberately deferred as out of scope for this submission, consistent with the locked project scope documented in Chapter One. They are recommended as the basis for future development:

- Dark Mode Toggle: A user-selectable dark colour theme, applied consistently across all calendar, dashboard, and panel views.
- Weekly Digest Push Notification: An automated Monday 8am summary push notifying a student of the total number of lectures and TMA deadlines for the coming week.
- Recurring Lecture Templates: An administrator-facing feature to define a repeating weekly lecture pattern, automatically generating individual lecture rows for the full facilitation window rather than requiring each session to be created or imported individually.
- Study Material Links per Lecture: The ability to attach a link to relevant eLearn course material directly on a lecture record, surfaced in the LectureDetailSheet.
- Attendance Data Export: A CSV export of a student's personal attendance history across all enrolled courses for a given academic session.
- Past Lectures Archive: A dedicated archive view listing all lectures whose end_time has passed, with quick access to any attached recording link.
- NOUN Study Centre Filter: An administrative filter allowing the timetable and reporting views to be scoped to a specific study centre.

In addition to these explicitly deferred features, three further directions are recommended for future research and development beyond the original scope boundary: first, a final closing of the three pending Definition of Done items through scheduled, supervised on-device testing immediately following submission; second, an extension of the attendance self-tracking feature to support lecturer-side attendance verification or cross-referencing against Zoom's own participant logs; and third, exploration of a native mobile wrapper (for example, via Capacitor) for institutions or students whose devices do not support the Web Push API at all, such as users on iOS versions earlier than 16.4.

REFERENCES

Adewale, O. S., Adaora, N. F., & Uchenna, E. K. (2021). Challenges of distance learning in Nigerian open universities: A survey of the National Open University of Nigeria. Journal of Open and Distance Learning in Africa, 3(1), 14–27.

Agbo, C., Nwosu, A., Eze, F., & Okafor, G. (2022). Design and implementation of an automated lecture attendance and notification system using Android and Firebase Cloud Messaging. International Journal of Computer Science and Information Technology, 14(3), 55 –68.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. Retrieved from https://agilemanifesto.org

Biorn-Hansen, A., Majchrzak, T. A., & Gronli, T. M. (2017). Progressive web apps: The possible web-native unifier for mobile development. Proceedings of the 13th International Conference on Web Information Systems and Technologies (WEBIST 2017), 344–351. https://doi.org/10.5220/0006353703440351

Connolly, T., & Begg, C. (2015). Database systems: A practical approach to design, implementation, and management (6th ed.). Pearson Education.

Dodds, K. (2020). Epic React: The beginner's guide to React. Retrieved from https://epicreact.dev

Laudon, K. C., & Laudon, J. P. (2025). Management information systems: Managing the digital firm (18th ed.). Pearson Education.

Nwachukwu, C. E., & Emeka, O. (2020). WhatsApp-based automated reminder system for academic activities in Nigerian tertiary institutions. West African Journal of Information Technology, 8(2), 33–44.

O'Brien, J. A., & Marakas, G. M. (2020). Management information systems (10th ed.). McGraw-Hill Education.

Oyelaran, O. A., & Okike, E. U. (2019). SMS-based notification systems for academic communities in Sub-Saharan Africa: Opportunities and constraints. African Journal of Computing and ICT, 12(1), 18–27.

Sadalage, P. J., & Fowler, M. (2013). NoSQL distilled: A brief guide to the emerging world of polyglot persistence. Addison-Wesley.

Supabase. (2024). Supabase documentation. Retrieved from https://supabase.com/docs

Thomson, M., Damaggio, E., & Beverloo, B. (2016). Generic event delivery using HTTP Push (RFC 8030). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc8030

Thomson, M., & Beverloo, B. (2017). Voluntary Application Server Identification (VAPID) for Web Push (RFC 8292). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc8292

APPENDIX A: SOURCE CODE LISTINGS

This appendix presents selected source code listings for the key configuration, authentication, and security files of the Smart Lecture Reminder. The complete source code is available in the project repository.

A.1 Supabase Client Configuration (src/services/supabase.js)

This file initializes the Supabase client using environment variables and exports the client instance used throughout the application.

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

A.2 Authentication Context (src/contexts/AuthContext.jsx)

This file provides global authentication state to the application using React Context, listening for Supabase Auth state changes and fetching the corresponding users row on every change.

```
import { createContext, useContext, useEffect, useState } from 'react'
import { supabase } from '../services/supabase'

const AuthContext = createContext(null)
export const useAuth = () => useContext(AuthContext)

export function AuthProvider({ children }) {
 const [user, setUser] = useState(null)
 const [profile, setProfile] = useState(null)
 const [loading, setLoading] = useState(true)

 useEffect(() => {
 supabase.auth.getSession().then(({ data: { session } }) => {
 setUser(session?.user ?? null)
 if (session?.user) fetchProfile(session.user.id)
 setLoading(false)
 })
 const { data: { subscription } } = supabase.auth.onAuthStateChange(
 (_event, session) => {
 setUser(session?.user ?? null)
 if (session?.user) fetchProfile(session.user.id)
 }
 )
 return () => subscription.unsubscribe()
 }, [])
```

```
 async function fetchProfile(userId) {
 const { data } = await supabase.from('users').select('*').eq('id',
userId).single()
 setProfile(data)
 }

 return (
 <AuthContext.Provider value={{ user, profile, loading }}>
 {children}
 </AuthContext.Provider>
 )
}
```

A.3 Push Subscription Utility (src/services/push.service.js)

This file subscribes the current device to Web Push using the VAPID public key and appends the resulting subscription object to the student's push_subscriptions array in Supabase.

```
import { supabase } from './supabase'

export async function subscribeToPush(userId) {
 const registration = await navigator.serviceWorker.ready
 const subscription = await registration.pushManager.subscribe({
 userVisibleOnly: true,
 applicationServerKey: import.meta.env.VITE_VAPID_PUBLIC_KEY,
 })
 const { data: user } = await supabase
 .from('users').select('push_subscriptions').eq('id', userId).single()
 const updated = [...(user?.push_subscriptions || []), subscription.toJSON()]
 await supabase.from('users').update({ push_subscriptions: updated }).eq('id',
userId)
 return subscription
}
```

A.4 Row Level Security Policy Excerpt (Supabase SQL)

This excerpt shows the Row Level Security policies applied to the lectures table, restricting write access to administrators and to lecturers updating only their own assigned lectures.

```
-- Lectures: all authenticated users read; admin or assigned lecturer writes
alter table public.lectures enable row level security;

create policy "All read lectures" on public.lectures
 for select using (auth.role() = 'authenticated');

create policy "Admin writes lectures" on public.lectures
 for all using (
 exists (select 1 from public.users where id = auth.uid() and role = 'admin')
 );

create policy "Lecturer updates own lectures" on public.lectures
 for update using (facilitator_id = auth.uid());
```

APPENDIX B: SYSTEM INSTALLATION AND DEPLOYMENT GUIDE

B.1 Prerequisites

- Node.js v20 LTS and npm
- A free Supabase account and project (supabase.com)
- A free Netlify account connected to the project's GitHub repository
- A generated VAPID key pair (npx web-push generate-vapid-keys)

B.2 Local Installation

Clone the repository, run npm install, copy .env.example to .env.local, and populate VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, and VITE_VAPID_PUBLIC_KEY. Set Edge Function secrets via `npx supabase secrets set`. Run the SQL migration files in supabase/migrations/ in order, then run npm run dev.

B.3 Production Build

Run npm run build to produce the production bundle in the dist/ directory. The build must complete with zero errors and zero broken imports before deployment.

B.4 Netlify Deployment

Connect the GitHub repository to a new Netlify site. Set the build command to npm run build and the publish directory to dist. No environment variables are required in Netlify it serves the static build only. Deploy Edge Functions via `npx supabase functions deploy send-reminders` and `npx supabase functions deploy cancel-notify`, then set secrets via `npx supabase secrets set`. Confirm the Supabase Cron job under Database → Cron Jobs. The system is live at https://smart-lecture-reminder-noun.netlify.app.

APPENDIX C: USER MANUAL

C.1 Logging In

Navigate to the deployed application URL. Enter your registered email and password on the Login page and click Log In. If you have forgotten your password, click "Forgot password?" to receive a reset link by email.

C.2 Student: Completing Onboarding

After registering as a student, you will be guided through a four-step Onboarding wizard. Select your programme and level, then upload your course registration slip (a digital PDF works best for automatic course detection). Review the pre-ticked courses, adjust if needed, and confirm enrollment. Grant push notification permission when prompted, and optionally install the application to your home screen.

C.3 Student: Viewing Your Calendar and Marking Attendance

Use the Calendar tab to switch between Month, Week, and Day views. Tap any lecture to view its details, join the Zoom session, or for past lectures mark your attendance as attended, missed, or excused.

C.4 Lecturer: Managing Your Lectures

After your account is approved by an administrator and courses are assigned to you, log in to view only your assigned lectures. Use the Reschedule or Cancel buttons on any lecture to notify enrolled students instantly, or use the Announcement or Add Recording options to share updates.

C.5 Admin: Managing the Timetable

Use the Admin panel’s Lectures tab to create, edit, or delete individual lectures, or use Bulk Import to upload a CSV timetable. Use the TMA tab to set assignment deadlines, the Users tab to approve pending lecturers and manage accounts, and the Settings tab to configure the active academic session.

C.6 Managing Notification Preferences

From the Settings page, toggle individual notification types (reminders, cancellations, announcements, TMA deadlines) on or off, and use the "Send test notification" option to confirm push delivery is working on your device.

APPENDIX D: TEST RESULTS SUMMARY

This appendix summarizes the automated test coverage for the project. The full end-to-end suite, covering admin, lecturer, student, and cross-role scenarios, recorded 27 passing test cases out of 27 at the time of writing. Unit and integration test results in full are presented in Tables 4.2 and 4.3 of Chapter Four, and the Definition of Done checklist status is presented in Table 4.4.

APPENDIX E: SAMPLE DATA

E.1 Sample Course Registration Slip Extraction

A sample digital PDF course registration slip containing the text "CIT 403 Research Methods in Computing, CIT 411 Expert Systems, GST 305 Communication in English" was processed by slipParser.js and correctly returned the array ["CIT 403", "CIT 411", "GST 305"].

E.2 Sample Lecture Record

A representative lecture record: course_code "CIT 403", facilitator_name "Dr. Fatima Ibrahim", start_time "2026-07-21T10:00:00", meeting_web_url "https://zoom.us/j/8000123456", status "scheduled", notifications_sent {"30min": false, "15min": false, "5min": false}.

E.3 Sample Push Notification Payload

A representative 5-minute reminder payload dispatched by send-reminders/index.ts: {"title": "CIT 403 is about to start", "body": "Facilitator: Dr. Fatima Ibrahim | Tap to join", "lectureId": "<uuid>", "meetingWebUrl": "https://zoom.us/j/8000123456"}.